Student Feedback Management System

A Comprehensive Project Report

Submitted in partial fulfillment of the requirements for the course:
Object-Oriented Software Engineering (OOSE) Lab

Field	Details
Name	T. Aum Namaha
Reg. No.	2023001973
Section	AIML B
Group	9
Subject	Object-Oriented Software Engineering
Instructor	Dr. Phani Sheetal
Date	31/10/2026

Department of Computer Science & Engineering / Artificial Intelligence & Machine Learning
GITAM University, Hyderabad
Academic Year: 2026-2027

Certificate

This is to certify that **T. Aum Namaha**, Reg. No. **2023001973**, of **AIML B**, Department of Computer Science & Engineering / Artificial Intelligence & Machine Learning, **GITAM University, Hyderabad**, has satisfactorily completed the project **"Student Feedback Management System"** for the course **Object-Oriented Software Engineering (OOSE) Lab** during the academic year **2026-2027**.

Project Guide / Instructor: _____ Dr. Phani Sheetal

Head of Department: _____

Date: _____ 31/10/2026

Declaration

I hereby declare that the work presented in this project report entitled **“Student Feedback Management System”** is an original effort carried out by me, **T. Aum Namaha** (Reg. No. **2023001973**), as part of the course requirements for Object-Oriented Software Engineering (OOSE) Lab at **GITAM University, Hyderabad**.

The material embodied in this report has not been submitted to any other institution or university for the award of any degree or diploma. All sources of information and references used in this work have been duly acknowledged and cited herein.

Signature: _____

Name: T. Aum Namaha

Reg. No.: 2023001973

Date: 31/10/2026

Table of Contents

Chapter 1: Introduction ... 7
1.1 Overview of Feedback Systems in Educational Institutions ... 7
1.2 The Need for a Structured Feedback Management System ... 7
1.3 Relevance to Object-Oriented Software Engineering ... 7
Chapter 2: Problem Statement ... 8
2.1 Identified Problems ... 8
2.2 Problem Scope ... 8
Chapter 3: Objectives ... 9
3.1 Primary Objective ... 9
3.2 Specific Objectives ... 9
Chapter 4: Existing System ... 10
4.1 Current Feedback Collection Practices ... 10
4.2 Comparative Analysis ... 10
4.3 Summary of Gaps ... 11
Chapter 5: Proposed System ... 12
5.1 Overview ... 12
5.2 Key Features ... 12
5.3 System Workflow ... 13
Chapter 6: Scope ... 14
6.1 In-Scope Features ... 14
6.2 Out-of-Scope Features ... 14
6.3 Target Users ... 14
Chapter 7: Functional Requirements ... 15
Chapter 8: Non-Functional Requirements ... 19
Chapter 9: System Architecture ... 21
9.1 Architectural Pattern — Three-Tier Architecture ... 21
9.2 Detailed Layer Descriptions ... 22
9.3 Design Patterns Used ... 22
Chapter 10: Use Case Diagram ... 24
10.1 Overview and Description ... 24
Chapter 11: Class Diagram ... 26
11.1 Overview and Description ... 26
Chapter 12: Sequence Diagrams ... 29
12.1 Sequence for "Submit Feedback" (Student) ... 29
12.2 Sequence for "Faculty Login and Dashboard Load" ... 30
12.3 Post Comment (Faculty, AJAX) ... 31
12.4 Verify Resolved Feedback (Faculty or Admin) ... 32
12.5 Admin Update Feedback Status ... 34
Chapter 13: Activity Diagrams ... 35
13.1 Feedback Status Lifecycle (Activity Flow) ... 35
13.2 Student Registration Activity Flow ... 36
13.3 Admin Filtering Activity Flow ... 37
Chapter 14: ER Diagram ... 39
14.1 Overview and Description ... 39
Chapter 15: Database Design ... 43
15.1 Schema Overview ... 43
15.2 Table: users ... 43
15.3 Table: faculty ... 43
15.4 Table: feedback ... 44
15.5 Table: comment ... 44
15.6 Status Enum Details ... 44
15.7 Normalization Analysis ... 45
15.8 Data Integrity Constraints ... 45
Chapter 16: UI Design ... 46
16.1 Design Philosophy and Visual Language ... 46
16.2 Key Screen Descriptions ... 46
Chapter 17: Implementation ... 50
17.1 Backend Architecture — Flask Application Factory ... 50
17.2 Database Models — SQLAlchemy ORM ... 50
17.3 Authentication Module (`blueprints/auth/routes.py`) ... 51
17.4 Student Module (`blueprints/student/__init__.py`) ... 51
17.5 Admin Module (`blueprints/admin/__init__.py`) ... 51
17.6 Faculty Module (`blueprints/faculty/routes.py`) ... 52
17.7 Frontend Implementation — Templates and Static Assets ... 52
17.8 Configuration Management (`config.py`) ... 52
Chapter 18: Testing ... 53
18.1 Testing Methodology ... 53
18.2 Test Files Structure ... 53
18.3 Test Cases Table ... 53

18.4 Security Test Results ... 55
Chapter 19: Results ... 56
19.1 System Completion Status ... 56
19.2 Functional Verification Summary ... 56
19.3 Performance Notes ... 56
19.4 Screenshots Placeholder ... 56
19.5 Achieved Outcomes ... 58
Chapter 20: Advantages ... 59
20.1 Structural and Operational Advantages ... 59
20.2 Technical Advantages ... 59
Chapter 21: Limitations ... 60
21.1 Functional Limitations ... 60
21.2 Technical Limitations ... 60
21.3 Operational Limitations ... 60
Chapter 22: Future Enhancements ... 61
22.1 Short-Term Enhancements (Feasible in Current Architecture) ... 61
22.2 Medium-Term Enhancements ... 61
22.3 Long-Term Enhancements ... 61
Chapter 23: Conclusion ... 62
23.1 Project Summary ... 62
23.2 Application of OOSE Principles ... 62
23.3 Learning Outcomes and Reflections ... 62
Chapter 24: References ... 63
24.1 Technical Documentation ... 63
24.2 Software Engineering Textbooks ... 63
24.3 Web Development Resources ... 63
24.4 Security References ... 63

Abstract

The Student Feedback Management System is a web-based application designed to streamline the process of collecting, managing, and resolving student feedback regarding various aspects of campus life. Traditional paper-based or informal digital feedback mechanisms often suffer from issues such as data loss, lack of categorization, inability to track resolution status, and limited analytical capabilities. This project addresses these challenges by providing a structured, role-based platform where students can submit categorized feedback with ratings and optional anonymity, while administrators gain powerful tools for filtering, tracking, updating, and analyzing all incoming feedback.

The system is built using the Flask microframework in Python for the backend, MySQL as the relational database management system, and vanilla HTML5/CSS3/JavaScript for the frontend. The application follows Object-Oriented Software Engineering (OOSE) principles throughout its design and implementation lifecycle. Three user roles are supported — Student (submits feedback), Admin (manages all feedback, verifies resolutions, manages faculty accounts), and Faculty (reviews department-specific feedback, updates status, posts comments, and verifies resolved items) — with strict role-based access control ensuring that each user can only perform actions appropriate to their privileges. A critical security feature of this system is the enforcement of anonymous feedback privacy at the database query layer, not merely through UI-level hiding, ensuring that student identities remain protected even from administrative queries when anonymity is requested.

The project encompasses the complete software development lifecycle — from requirement analysis and system design (including use case, class, sequence, activity, and ER diagrams) through implementation, testing, and documentation. The modular architecture using Flask blueprints ensures maintainability and scalability, while SQLAlchemy ORM provides clean object-relational mapping for database interactions. Testing was conducted manually across all user flows to verify functional correctness, security compliance, and usability.

This report presents a comprehensive account of the project, covering problem identification, proposed solution design, architectural decisions, implementation details, testing methodology and results, advantages, limitations, future enhancements, and conclusions drawn from the development experience. The system serves as both an academic deliverable for the OOSE lab course and a functional prototype demonstrating practical application of software engineering principles in building a real-world web application.

Chapter 1: Introduction

1.1 Overview of Feedback Systems in Educational Institutions

Feedback is one of the most critical mechanisms through which educational institutions maintain quality, address student concerns, and continuously improve their services. Campus life encompasses numerous dimensions — food quality in dining facilities, teaching effectiveness of faculty members, infrastructure conditions including classrooms and laboratories, organization of cultural and academic events, and various other aspects that directly impact the student experience. When students have concerns or suggestions regarding any of these areas, they require an accessible, structured, and accountable channel through which to express their views.

Historically, many educational institutions in India and around the world have relied on paper-based feedback forms, suggestion boxes, or informal digital channels such as WhatsApp groups and email chains to collect student feedback. While these methods provide some mechanism for gathering input, they suffer from significant limitations that prevent effective action on the feedback received. Paper forms are prone to physical loss, difficult to categorize systematically, nearly impossible to track over time, and offer no real-time visibility into whether a concern has been addressed or is being processed. Digital alternatives like WhatsApp groups lack structure — important concerns get buried in casual conversation, there is no standardized format for submission, and administrators have no systematic way to filter, prioritize, or update the status of individual items.

1.2 The Need for a Structured Feedback Management System

The fundamental gap that this project addresses is the absence of an integrated, digital platform that provides end-to-end management of student feedback — from initial submission through categorization, tracking, resolution, and analysis. A well-designed system should not merely collect feedback but should create a transparent workflow where students can see the status of their submissions and administrators can efficiently manage, filter, and act upon them.

The Student Feedback Management System bridges this gap by providing a role-based web application that handles the complete feedback lifecycle. Students benefit from an intuitive interface to submit structured feedback with ratings and optional anonymity, while also being able to track the resolution progress of their own submissions. Administrators gain access to a comprehensive dashboard that supports multi-criteria filtering, keyword search, status management workflows, and aggregated analytics — all essential tools for making data-driven decisions about campus improvements.

1.3 Relevance to Object-Oriented Software Engineering

This project is designed as a practical application of Object-Oriented Software Engineering (OOSE) principles. OOSE emphasizes organizing software systems around objects that encapsulate both data and behavior, promoting modularity, reusability, maintainability, and scalability. In the context of this feedback management system, these principles manifest in several ways:

- **Encapsulation**: The `User` and `Feedback` classes encapsulate relevant attributes (such as name, email, password hash for users; category, rating, comment, status for feedback) along with associated behaviors (validation, relationship navigation).
- **Modularity through Blueprints**: Flask blueprints provide a structural mechanism to separate concerns — authentication logic is isolated from student-facing features and administrative operations.
- **Relationship Management**: The one-to-many relationship between users and feedback records demonstrates how objects relate to each other in a real system, mirroring the entity relationships found in database design.
- **Design Patterns**: The application factory pattern for creating Flask applications and the blueprint pattern for organizing routes follow established software engineering best practices that promote testability and scalability.

By implementing these principles in a tangible, functional web application, this project provides practical experience with OOSE concepts that complement theoretical learning from lectures and textbooks.

Chapter 2: Problem Statement

2.1 Identified Problems

Educational institutions face several persistent challenges in their feedback collection and management processes:

Problem 1 — Lack of Structured Collection: Paper-based or informal digital feedback mechanisms do not enforce a consistent format for submissions. Students may describe concerns in unstructured text without providing quantifiable ratings, making it difficult to perform quantitative analysis or prioritize issues based on severity. Without standardized categories (such as Food, Faculty, Infrastructure, Events), administrators cannot efficiently sort and route feedback to the appropriate department or team responsible for resolution.

Problem 2 — No Tracking Mechanism: Once a paper form is submitted into a suggestion box or an informal message is sent through a chat group, there is no systematic way to track whether it has been received, reviewed, acted upon, or resolved. Students receive no feedback on the status of their submissions, leading to frustration and reduced participation in future feedback cycles. Administrators lack visibility into how many pending items exist, which ones are overdue for resolution, and what percentage of concerns have been addressed within a given timeframe.

Problem 3 — Limited Analytical Capabilities: Without digitized, structured data, institutions cannot generate meaningful reports or trends from the feedback they collect. Questions such as “Which category has received the most complaints this month?” or “Has food quality feedback improved over the semester?” remain unanswerable because the underlying data either does not exist in a queryable format or is scattered across physical files and informal digital channels.

Problem 4 — Privacy Concerns for Sensitive Feedback: Students who wish to provide honest feedback about sensitive topics (such as faculty behavior, harassment concerns, or institutional policies) may be reluctant to do so if their identity will be known to administrators. Traditional systems rarely offer an anonymity option, which can lead to self-censorship and a loss of valuable critical insights that institutions need to improve.

Problem 5 — Administrative Overhead: Manual sorting, categorization, and status tracking of paper-based feedback requires significant human effort. Administrators spend considerable time physically organizing documents, manually noting down concerns in registers or spreadsheets, and communicating status updates through informal channels. This overhead reduces the efficiency with which genuine issues can be identified and resolved.

2.2 Problem Scope

The problem scope is limited to feedback related to campus life within a single educational institution. The system does not address academic grading disputes, formal complaints through official grievance redressal mechanisms, or external stakeholder feedback (such as from parents or alumni). The target users are enrolled students and administrative staff of the institution. The geographic and operational scope is confined to a local deployment suitable for campus use, with plans for potential future scaling to multi-campus environments.

Chapter 3: Objectives

3.1 Primary Objective

To design, develop, and document a fully functional web-based Student Feedback Management System that provides students with an accessible platform to submit structured feedback about campus life and equips administrators with comprehensive tools to manage, track, categorize, filter, and analyze all incoming feedback — thereby creating a transparent, accountable, and data-driven feedback resolution workflow.

3.2 Specific Objectives

The following specific objectives guide the design and implementation of the system:

1. **Implement Secure Authentication**: Provide role-based registration and login functionality for students (self-registration with roll number and email) and administrators (manual seeding), using secure password hashing through werkzeug.security and session-based authentication management.

2. **Enable Structured Feedback Submission**: Allow students to submit feedback with a standardized format including category selection (Food, Faculty, Infrastructure, Events, Other), numerical rating (1–5 scale), free-text comments, and an optional anonymity toggle that suppresses student identity at the query layer for admin-facing views.

3. **Develop Student Dashboard**: Create a personalized dashboard where logged-in students can view their own feedback history, including submission details, current status badges (Pending/In Progress/Resolved), and timestamps — providing transparency about the progress of their concerns.

4. **Build Admin Management Interface**: Design an admin-only dashboard that displays all student feedback in a tabular format with multi-criteria filtering capabilities (by category, rating range, status, date range) and keyword search functionality for efficient identification of specific concerns within large datasets.

5. **Implement Status Workflow**: Enable administrators to update the status of individual feedback entries through a defined progression: Pending → In Progress → Resolved, with automatic timestamping of each status change in the database's `updated_at` column.

6. **Generate Analytical Reports**: Provide an admin-accessible reports page that displays aggregated statistics including average rating per category, total feedback count per category, and temporal trends (weekly/monthly submission patterns) to support data-driven decision-making.

7. **Enforce Privacy Through Query-Level Anonymity**: Ensure that anonymous feedback submissions are handled with identity suppression at the database query layer — not merely through UI-level hiding — so that student identities remain protected even if templates or future features are modified without corresponding privacy updates.

8. **Enable Faculty Review Workflow**: Allow faculty members to log in with department-specific access, view only feedback matching their department (excluding 5-star ratings), update status through Pending → In Progress → Resolved, post comments on feedback items, and verify resolved items as Verified/Closed or revert them to In Progress on failure.

9. **Implement Comment Threads**: Support Reddit-style threaded discussions on each feedback item where matching faculty can post comments and submitting students can view read-only, with self-referencing parent IDs for nested replies.

10. **Add Verification and Escalation Workflow**: Enable both admin and faculty to verify resolved feedback (Verified/Closed on success, revert to In Progress with failed counter increment on failure). Automatically pin overdue items as escalated and flag items exceeding the failed verification threshold.

11. **Apply OOSE Principles**: Demonstrate practical application of object-oriented software engineering concepts including class design, encapsulation, modular architecture (blueprints), relationship management, and the complete software development lifecycle throughout the project.

12. **Ensure Maintainability and Scalability**: Structure the codebase using Flask blueprints, separated models, organized templates, and a clean folder hierarchy to ensure that future developers can easily understand, extend, and maintain the system as requirements evolve.

Chapter 4: Existing System

4.1 Current Feedback Collection Practices

To understand the motivation behind this project, it is essential to examine how feedback collection typically operates in educational institutions currently. Across many colleges and universities — particularly in India — several common approaches exist for gathering student feedback on campus life:

4.1.1 Paper-Based Suggestion Boxes

The most traditional method involves physical suggestion boxes placed in common areas of the campus (hostels, canteens, libraries). Students write their concerns on paper slips and deposit them into the box. These suggestions are periodically retrieved by administrative staff who manually read through each entry, categorize them informally, and attempt to route relevant items to appropriate departments.

Limitations: - **Physical Loss Risk**: Paper slips can be lost, damaged, or misfiled. There is no backup mechanism to recover feedback that was submitted but never retrieved. - **No Standardization**: Students write in free-form text without any structured fields (category, rating), making it impossible to perform quantitative analysis. - **Poor Discoverability**: Searching through physical papers for a specific concern or keyword requires manual reading of every entry — an impractical task with large volumes. - **No Status Tracking**: Once a paper slip is deposited, the student has no way of knowing whether it was read, acted upon, or resolved. There is no reference number or tracking ID. - **Time Consumption**: Manual sorting and categorization by administrative staff is labor-intensive and delays response times significantly.

4.1.2 Informal Digital Channels (WhatsApp Groups, Email Chains)

Many institutions have moved partially toward digital channels, using WhatsApp groups, Telegram channels, or email addresses for students to submit feedback informally. While this eliminates the physical storage problem of paper-based systems, it introduces new challenges:

Limitations: - **Unstructured Data**: Messages arrive in arbitrary formats with no standardized fields. Important details may be missing (e.g., which building has a broken AC, what specific food item is problematic). - **Information Overload**: In large groups, genuine feedback messages get buried under casual conversation, memes, or off-topic discussions. Critical concerns become difficult to identify. - **No Filtering or Search**: While WhatsApp and email clients have basic search functionality, they lack structured filtering (e.g., “show me all food-related complaints from this month”). - **Identity Exposure**: Students sending messages through their personal accounts reveal their identity automatically — there is no option for anonymous feedback. - **No Workflow Management**: There is no mechanism to update the status of a concern or notify the student about progress.

4.1.3 Annual Student Feedback Forms (Academic)

Many institutions conduct annual or semester-end feedback forms, typically focused on evaluating teaching effectiveness and course content. These are often conducted through paper surveys or basic online forms administered by the academic cell.

Limitations: - **Limited Scope**: These forms focus primarily on academic evaluation rather than broader campus life concerns (food quality, hostel conditions, event organization). - **Infrequent Collection**: Being annual or semester-based, they do not provide real-time feedback channels for urgent issues that arise between formal evaluation periods. - **No Resolution Tracking**: Students submit their evaluations but rarely receive information about whether any changes were made as a result of their feedback.

4.2 Comparative Analysis

Aspect	Paper-Based	Informal Digital	Academic Forms Only	Proposed System
Structured Format	No	No	Partial	Yes (category, rating)
Data Storage	Physical papers	Chat logs/Emails	Paper/PDF	MySQL Database
Search & Filter Capability	None	Basic text search	Limited	Multi-criteria filtering
Status Tracking	No	No	No	Yes (Pending → Resolved)
Anonymity Support	Partial (handwriting)	No	No	Yes (query-level)
Analytics & Reports	None	None	Basic summaries	Aggregated statistics
Accessibility	Physical presence	Internet required	Scheduled events	Any browser, anytime
Data Loss Risk	High	Medium	Medium	Low (database-backed)

4.3 Summary of Gaps

The existing systems collectively fail to provide a comprehensive solution that addresses the full feedback lifecycle — from submission through tracking to resolution and analysis. The proposed Student Feedback Management System fills these gaps by integrating all necessary capabilities into a single, structured, secure, and accessible platform built on modern web technologies.

Chapter 5: Proposed System

5.1 Overview

The Student Feedback Management System is a three-tier web application that provides end-to-end management of student feedback about campus life. Built using Flask (Python) with MySQL or SQLite (fallback) databases, and vanilla frontend technologies, the system implements role-based access control with distinct interfaces for Students and Administrators. The architecture follows Object-Oriented Software Engineering principles, with modular code organization through Flask blueprints, clean database design using SQLAlchemy ORM, and separation of presentation, application, and data layers.

5.2 Key Features

5.2.1 Student-Facing Features

- **Self-Registration**: Students create accounts by providing their name, roll number (unique), email address (unique), and a password that is hashed before storage using werkzeug.security's bcrypt-style hashing algorithm.
- **Secure Login**: Authentication validates credentials against the database and establishes a server-side session containing the user ID, role, and name — enabling persistent login across page requests without exposing credentials.
- **Feedback Submission Form**: A structured form requiring category selection (dropdown with five options), numeric rating (1–5 star selector), free-text comment field, and an optional anonymity checkbox. All inputs are validated server-side before database insertion.
- **Personal Dashboard**: Displays all feedback submitted by the logged-in student in a chronological table showing category badge, numerical rating, comment preview, status indicator with color coding, and submission timestamp.

5.2.2 Admin-Facing Features

- **Admin Login**: Separate authentication path for administrators who are pre-seeded into the database via a dedicated seed script (`seed.py`). Admin accounts cannot be self-created — they must be provisioned by an existing administrator or developer.
- **Faculty Account Management**: Admins can list all faculty accounts and create new ones with name, email, faculty ID, department, subject taught, and password. Faculty IDs are unique constraints preventing duplicate accounts.
- **Master Feedback Dashboard**: A comprehensive table displaying all student feedback entries with columns for ID, submitter name (or "Anonymous"), category badge, rating stars, comment preview, status badge, and submission date. Supports multi-criteria filtering through a filter bar containing dropdown menus for category and status, a numeric input for minimum rating, date range pickers, and a keyword search text field.
- **Individual Feedback Management**: Clicking on any feedback entry opens a detail view showing the complete comment (not just preview), submitter information (suppressed if anonymous), all metadata fields, and controls to update the status through the Pending → In Progress → Resolved workflow.
- **Verification Queue**: A dedicated Faculty Review section on the admin dashboard shows Resolved items awaiting verification. Admins can mark them Verified/Closed (success) or fail verification (reverts to In Progress with failed counter increment). Escalated items (Pinned + high-fail-count) are highlighted.
- **Reports & Analytics Page**: Displays aggregated visualizations: average rating per category, feedback count distribution across categories and departments, monthly submission trends, resolution time metrics, and escalation counts.

5.2.3 Faculty-Facing Features

- **Faculty Login**: Faculty members authenticate using email and password. Accounts are seeded by admins (no self-registration) with department and subject_taught attributes that determine which feedback they can access.
- **Department-Filtered Dashboard**: Displays only feedback from the faculty member's department, excluding 5-star ratings. Items are prioritized by subject match, pinned status, and escalation flags. Shows countdown timers for review deadlines.
- **Status Transitions**: Faculty can update their assigned department's feedback through Pending → In Progress → Resolved, with automatic tracking of which faculty resolved each item (resolved_by_faculty_id).
- **Comment Threads**: Matching faculty can post comments on any feedback in their department. Comments support nested replies via self-referencing parent IDs. Students viewing their own feedback see read-only comment threads.
- **Resolution Verification**: Faculty can verify resolved items by marking them Verified/Closed (success) or reverting to In Progress with a failed verification counter increment (failure).

5.2.4 Admin-Facing Faculty Management

- **Password Hashing**: All passwords are hashed using werkzeug.security before storage. The hashing process includes automatic salt generation and uses bcrypt-compatible algorithms that are computationally expensive to reverse, protecting against database breach scenarios.
- **Session Management**: Flask sessions use cryptographically signed cookies with an application-specific secret key. Session data (user ID, role) is stored server-side references while the cookie itself only contains a signature that cannot be tampered with without detection.
- **Role-Based Access Control**: Decorator-based route protection (`@login_required`, `@admin_required`) ensures that students cannot access admin routes and vice versa. Every protected route checks session state before executing any logic.
- **Anonymous Feedback Privacy**: Student identity suppression is enforced at the serialization layer. The `to_admin_dict()` method returns "Anonymous" for anonymous rows regardless of whether `self.author` is loaded; the `to_faculty_dict()` method only exposes class/year derived from roll number, never name/email/roll. Admin queries do not join the users table for anonymous feedback.
- **CSRF Protection**: All POST forms include hidden CSRF tokens generated by Flask-WTF's CSRFProtect extension. AJAX comment submissions send the token via `X-CSRFToken` headers read from a `<meta>` tag in the base template layout.

5.3 System Workflow

The system follows a clear linear workflow for each feedback item, involving three roles:

1. **Student registers** → Account created with hashed password in `users` table (role='student')
2. **Student logs in** → Session established, redirected to student dashboard
3. **Student submits feedback** → Record inserted into `feedback` table with status='Pending', current timestamp, and review_deadline set to 24h from now
4. **Admin reviews and filters** → Admin applies multi-criteria filters on admin dashboard to find relevant items
5. **Faculty reviews department feedback** → Matching faculty (by department) sees non-5-star items on their dashboard with subject-match highlighting
6. **Status transitions** → Admin or faculty updates status: Pending → In Progress → Resolved; resolved_by_faculty_id recorded when faculty marks Resolved
7. **Resolution verification** → Both admin and faculty can verify Resolved items: success → Verified/Closed, failure → In Progress with failed_verification_count incremented
8. **Escalation of overdue items** → Items past review_deadline are automatically pinned; items with failed_verification_count >= 3 are flagged as escalated
9. **Comment threads** → Matching faculty post comments on feedback items via AJAX; students view read-only
10. **Student views progress** → Student can see updated status, countdown timers, and comment threads on their personal dashboard
11. **Reports aggregate data** → Analytics computed from all feedback records for administrative review

This workflow ensures that every piece of student feedback enters a managed lifecycle with clear accountability and traceability across three roles — submission by students, review/verification by faculty and admin, and monitoring by students.

Faculty Account Provisioning

Faculty accounts are created exclusively by admins via the "Manage Faculty" page (`/admin/faculty/create`). Each account includes department and subject_taught assignments that determine which feedback items appear on their dashboard.

Chapter 6: Scope

6.1 In-Scope Features

The following features are within the scope of this project implementation:

- Student self-registration with roll number and email validation
- Multi-role login supporting students, admins, and faculty via a unified /login endpoint
- Faculty account management: admins can list all faculty accounts and create new ones (no faculty self-registration)
- Structured feedback submission with category, rating (1–5), comment text, and anonymity toggle
- Student dashboard displaying personal feedback history with status tracking
- Admin dashboard with full visibility of all feedback entries
- Multi-criteria filtering on the admin dashboard (category, rating, status, date range)
- Keyword search across feedback comments
- Faculty dashboard showing department-filtered feedback (excluding 5-star ratings) with subject-match highlighting and deadline countdowns
- Comment threads on each feedback item — matching faculty can post; students view read-only with nested reply support
- Resolution verification by both admin and faculty: Verified/Closed on success, revert to In Progress with failed counter increment on failure
- Automatic escalation of overdue feedback (Pinned status) with escalation deadlines
- Reports page with aggregated statistics (average ratings, counts per category/department, trends, escalation metrics)
- Anonymous feedback handling enforced at the query layer
- Responsive design suitable for desktop and mobile browsers
- Local deployment on a development server

6.2 Out-of-Scope Features

The following features are explicitly excluded from this project scope:

- **Multi-campus or multi-institution support**: The system is designed for a single institution with a single database instance.
- **Email notifications**: No automated email notification system is implemented to alert students when their feedback status changes. This could be added as a future enhancement using Flask-Mail or similar integration.
- **File/image attachments**: Students cannot upload photographs or documents along with their feedback submissions. Only text comments are supported.
- **Mobile application**: The system is web-based only. No native iOS or Android mobile app is developed in this project.
- **Real-time notifications**: There is no WebSocket or polling-based real-time notification system for admin alerts when new feedback arrives.
- **Third-party integrations**: Integration with existing institutional systems (ERP, student information systems, HR databases) is not included.
- **Multi-language support**: The interface is designed in English only; internationalization (i18n) is not implemented.
- **Advanced analytics beyond basic aggregations**: Predictive analytics, sentiment analysis of comments, or machine learning-based categorization are outside the scope of this project.
- **Public-facing feedback portal**: Only authenticated users (students and admins) can access the system. There is no guest or public submission capability.

6.3 Target Users

User Type	Description	Expected Count
Students	Enrolled students of the institution	50–500 (typical class/section size)
Admins	Administrative staff responsible for campus operations and faculty management	1–5
Faculty	Teaching staff who review department-specific feedback, update status, post comments, and verify resolutions	10–50

The system is designed to comfortably handle the expected user load with performance targets of dashboard loads within 2–3 seconds for hundreds of feedback records.

Chapter 7: Functional Requirements

Functional requirements describe what the system must do — the specific behaviors, functions, and capabilities that users expect from the application. Each requirement has been assigned a unique identifier for traceability throughout design, implementation, and testing phases.

FR-01: Student Registration

The system shall allow new students to register an account by providing their full name, roll number (which must be unique across all users), email address (which must also be unique), and a password (minimum 6 characters). Upon successful registration, the student shall be redirected to the login page with a success message. If the roll number or email already exists in the database, the system shall display an appropriate error message and prevent duplicate account creation.

Input: name (string), roll_number (string), email (string, format validated as valid email), password (string, minimum 6 characters)
Output: Account created in `users` table with role='student', hashed password, current timestamp; redirect to `/login` with flash message
Validation Rules: Roll number and email must be unique; password length ≥ 6; email format must match standard pattern

FR-02: Student Login

The system shall allow registered students to log in by providing either their email address or roll number along with their password. Upon successful authentication, a server-side session shall be created containing the user's ID, role ('student'), and name. The student shall be redirected to their personal dashboard (`/dashboard`). If credentials are invalid, an error message shall be displayed without revealing whether the identifier or password was incorrect (to prevent enumeration attacks).

Input: identifier (email or roll_number), password
Output: Session established; redirect to `/dashboard` on success; flash error message on failure
Security Note: Password is verified using `check_password_hash()` against stored hash — plaintext passwords are never compared directly

FR-03: Admin Login

The system shall allow administrators to log in using their email address and password. Unlike students, admin accounts cannot be self-created through the registration page; they must be provisioned manually via the seed script (`seed.py`) or direct database insertion. Upon successful authentication, a session is created with role='admin', and the user is redirected to the admin dashboard (`/admin/dashboard`).

Input: email (must match an admin account), password
Output: Session established; redirect to `/admin/dashboard` on success; flash error message on failure

FR-04: Feedback Submission

Authenticated students shall be able to submit feedback by selecting a category from the predefined list (Food, Faculty, Infrastructure, Events, Other), assigning a numerical rating between 1 and 5 (inclusive), entering a text comment of any length, and optionally toggling anonymous mode. Upon submission, the feedback record shall be inserted into the `feedback` table with the current timestamp as both `created_at` and initial `updated_at`, the default status 'Pending', and the logged-in student's ID linked as `student_id`. The student shall remain on the dashboard with a success confirmation message.

Input: category (ENUM), rating (TINYINT 1–5), comment (TEXT), is_anonymous (BOOLEAN)
Output: New row in `feedback` table; flash success message; page refresh showing new entry in history table
Database Constraint: Rating must be between 1 and 5 enforced by application validation

FR-05: Student Feedback History View

Authenticated students shall be able to view a paginated list of all feedback entries they have submitted. Each entry shall display the category badge, numerical rating (rendered as stars), comment preview (truncated to approximately 100 characters with "read more"

capability), current status indicator with color-coded badge (Pending = yellow/orange, In Progress = blue, Resolved = green), and submission timestamp.

Input: None (data retrieved from database based on logged-in user's ID)
Output: HTML table/list of feedback entries with all metadata fields displayed appropriately

FR-06: Admin Dashboard — Feedback Table Display

Authenticated administrators shall be able to view a comprehensive table of all student feedback entries in the system. Each row shall display the feedback ID, submitter name (or "Anonymous" if `is_anonymous=True`), category badge, rating stars, comment preview, status badge, and creation date. The table shall support pagination for large datasets.

Input: None (all data from `feedback` table joined with `users` table)
Output: Paginated HTML table; student name suppressed to "Anonymous" when `is_anonymous=True` at the query layer

FR-07: Admin Dashboard — Multi-Criteria Filtering

The admin dashboard shall provide filtering controls that allow administrators to narrow the displayed feedback entries by one or more criteria simultaneously: category (dropdown), status (dropdown), minimum rating (numeric input), and date range (start date and end date pickers). Filters shall be applied dynamically when submitted, updating the table to show only matching records.

Input: category filter (optional ENUM value), status filter (optional ENUM value), min_rating filter (optional TINYINT 1–5), start_date/end_date (optional DATE values)
Output: Filtered subset of feedback entries displayed in the table; count of matching results shown

FR-08: Admin Dashboard — Keyword Search

The admin dashboard shall include a keyword search text field that performs case-insensitive full-text matching against the comment content of all feedback entries. Results from keyword search can be combined with other filter criteria (category, status, rating, date range) for refined results.

Input: search_keyword (string)
Output: Feedback entries whose `comment` column contains the search keyword displayed in the table; supports combination with other filters

FR-09: Individual Feedback Detail View and Status Update

Authenticated administrators shall be able to click on any feedback entry to view its complete details, including full comment text (not truncated), submitter name, category, rating, anonymity status, creation timestamp, last update timestamp, and current status. From this detail view, the administrator can update the feedback status using a dropdown or button interface that progresses through: Pending → In Progress → Resolved. Each status change shall be recorded in the database with an updated `updated_at` timestamp.

Input: feedback_id (from URL), new_status (ENUM value)
Output: Updated `status` and `updated_at` fields in the `feedback` table; flash confirmation message; redirect back to admin dashboard or stay on detail page

FR-10: Reports Page — Aggregated Analytics

Authenticated administrators shall be able to access a reports page that displays aggregated statistics computed from all feedback records. The report shall include: (a) Average rating per category displayed as numerical values with bar chart visualization; (b) Total feedback count per category shown as a donut/pie chart with percentages; (c) Weekly or monthly submission trend line graph showing the number of feedback entries received over time, enabling identification of patterns and spikes in student concerns.

Input: None (data aggregated from `feedback` table using SQL GROUP BY and aggregate functions)
Output: HTML page with three visualization components displaying computed statistics; data calculated dynamically on each page load to reflect current database state

FR-11: Logout

Both students, administrators, and faculty members shall be able to log out of the system. Upon logout, the server-side session shall be destroyed (all session variables cleared), and the user shall be redirected to the login page (`/login`).

Input: Click “Logout” action
Output: Session cleared; redirect to `/login`; flash confirmation message

FR-12: Faculty Login

The system shall allow faculty members to log in using their email address and password. Unlike students, faculty accounts cannot be self-created through the registration page — they must be provisioned by an admin via the “Manage Faculty” page (`/admin/faculty/create`). Upon successful authentication, a session is created with role=‘faculty’ and the faculty member’s ID, redirecting to the faculty dashboard (`/faculty/dashboard`).

Input: email (must match a faculty account), password
Output: Session established with `faculty_id`; redirect to `/faculty/dashboard` on success; flash error message on failure

FR-13: Faculty Department-Filtered Dashboard

Authenticated faculty shall see only feedback entries that satisfy all of the following conditions: - The feedback’s `department` matches the logged-in faculty member’s `department`. - The feedback’s `rating` is not 5 (5-star ratings never reach faculty). When displaying results, student PII must be suppressed — only class and year derived from roll_number are shown (never name, email, or roll_number itself). Items matching the faculty’s subject_taught are prioritized first.

Input: None (data filtered server-side based on session `faculty_id`)
Output: HTML table of department-matching feedback with anonymized student info, status badges, countdown timers, and escalation indicators

FR-14: Faculty Status Update

Authenticated faculty shall be able to update the status of feedback items in their department through the workflow: Pending → In Progress → Resolved. When marking as Resolved, the system records `resolved_by_faculty_id` linking back to the faculty account. Each status change is timestamped.

Input: feedback_id (from URL), new_status (ENUM value)
Output: Updated `status`, `updated_at`, and optionally `resolved_by_faculty_id`; flash confirmation; redirect to detail page

FR-15: Comment Threads on Feedback

Matching faculty (those whose department matches the feedback’s department) shall be able to post comments on any feedback item in their department. Students can view comment threads read-only on their own submitted feedback. Comments support nested replies via self-referencing `parent_id`. AJAX-based posting sends JSON with CSRF headers.

Input: feedback_id, text (via POST form or JSON), optional parent_id for replies
Output: New Comment record inserted; real-time display of appended comment in thread

FR-16: Resolution Verification (Admin and Faculty)

Both admin and faculty shall be able to verify resolved feedback items via two actions: - **Verify Success**: Transitions status from ‘Resolved’ → ‘Verified/Closed’. Only allowed when current status is ‘Resolved’. - **Verify Failure**: Transitions status from ‘Resolved’ → ‘In Progress’, increments `failed_verification_count` by 1. Only allowed when current status is ‘Resolved’.

Input: feedback_id, action (‘success’ or ‘failure’)
Output: Status updated per verification outcome; failed counter incremented on failure; flash message confirming result

FR-17: Escalation of Overdue Feedback

The system shall automatically escalate overdue feedback by setting `status = 'Pinned'` and computing an `escalation_deadline` (3 days from detection). This occurs on each faculty dashboard load. Items are also escalated when `failed_verification_count >= 3`. Escalated items display prominently on both admin and faculty dashboards with warning badges.

Input: None (automatic, triggered on dashboard loads)
Output: Status changed to 'Pinned'; escalation_deadline set; visual escalation badge shown

FR-18: Admin Faculty Account Management

Admins shall be able to list all existing faculty accounts and create new ones. Creating a faculty account requires name, email, unique faculty_id, department, subject_taught, password (min 6 chars), and confirm_password. Duplicate email or faculty_id checks prevent conflicting accounts.

Input: GET `/admin/faculty` → list; POST `/admin/faculty/create` → create
Output: Table of all faculty accounts; on creation, new Faculty record with hashed password; flash confirmation

Chapter 8: Non-Functional Requirements

Non-functional requirements define how well the system performs its functions — quality attributes that govern performance, security, usability, reliability, and other operational characteristics. These requirements are equally important as functional ones because they determine whether the system is acceptable to end users in practice.

NFR-01: Usability

The user interface shall be clean, simple, and intuitive enough that a first-time user can navigate the system without formal training or documentation. The design shall follow consistent visual conventions — uniform color schemes for status badges (yellow/orange for Pending, blue for In Progress, green for Resolved), standardized button styles, and clear typography hierarchy. All interactive elements (buttons, links, forms) must have visible hover states and active states to provide user feedback during interaction.

Mobile Responsiveness: The layout shall adapt gracefully to different screen sizes using CSS media queries and flexible grid/flexbox layouts. Key breakpoints include: desktop (≥1024px), tablet (768–1023px), and mobile (<768px). On mobile devices, the admin filter bar should stack vertically, tables should allow horizontal scrolling or switch to card-based layout, and navigation menus should collapse into hamburger-style toggles.

Acceptance Criteria: A usability test with 5 representative users (3 students, 2 admins) shall result in at least 80% task completion rate without assistance across all core workflows (registration, login, feedback submission, filtering, status update).

NFR-02: Performance

The system shall provide responsive interactions under expected usage conditions. Dashboard pages, feedback detail views, and report computations should complete within a reasonable timeframe on standard development hardware (Flask debug server with SQLite or local MySQL).

Acceptance Criteria: The author is to measure actual page load times and computation durations during testing and record them in the Results chapter. Performance figures reported in Chapter 19 shall reflect measured values rather than targets, since no formal indexing strategy (beyond primary keys) or eager loading optimization has been implemented in the current codebase.

NFR-03: Security

The system shall implement multiple layers of security protection:

Password Storage: All user passwords shall be hashed using werkzeug.security's `generate_password_hash()` method with the 'pbkdf2:sha256' algorithm and a salt length of 64 bytes. This ensures that even in the event of a database breach, plaintext passwords cannot be recovered without significant computational effort.

Session Security: Flask sessions shall use cryptographically signed cookies (`SECRET_KEY`) to prevent tampering. Session data should ideally be stored server-side (using Flask-Session with Redis or file-based storage) for enhanced security, though cookie-based sessions are acceptable for this lab-scale deployment.

Role-Based Access Control: All admin-specific routes shall be protected by a decorator that verifies the user's session role equals 'admin'. Student-specific routes shall verify role equals 'student' (or simply require authentication). Unauthorized access attempts shall redirect to the login page with an appropriate flash message.

Anonymous Feedback Privacy: When `is_anonymous=True`, student identity suppression must occur at the serialization layer — specifically in the `to_admin_dict()` method which returns "Anonymous" for anonymous rows regardless of whether `self.author` is loaded. For faculty views, only class/year derived from roll_number is exposed (never name/email/roll). Admin queries do not eagerly load the users table for anonymous feedback.

CSRF Protection: All POST forms include hidden CSRF tokens generated by Flask-WTF's `CSRFProtect`. AJAX comment submissions send the token via `X-CSRFToken` headers read from a `<meta name="csrf-token">` tag in `base.html`. During testing, CSRF is disabled via `WTF_CSRF_ENABLED = False` in test configs.

SECRET_KEY Enforcement: In production (DEBUG=False), the application refuses to start if SECRET_KEY is not set via the environment variable — preventing accidental deployment with a default dev key.

SQL Injection Prevention: All database queries shall use SQLAlchemy ORM parameterized queries rather than raw SQL string concatenation, eliminating the risk of SQL injection attacks through user-supplied input.

Acceptance Criteria: (a) Database dump contains only hashed passwords, never plaintext; (b) Direct URL access to admin routes as a student redirects to login; (c) Anonymous feedback entries display "Anonymous" in all admin views regardless of template modifications; (d) Input fields accepting search keywords or form data do not produce SQL errors when containing special characters.

NFR-04: Reliability

The system shall ensure no data loss during feedback submission. Each database insert operation (new registration, new feedback entry, status update) shall be wrapped in proper transaction handling using `db.session.commit()` with appropriate error catching (`db.session.rollback()` on exception). If a database connection is lost mid-operation, the user shall see an appropriate error message rather than silently losing their input data.

Acceptance Criteria: (a) 100% of feedback submissions are persisted to the database under normal operating conditions; (b) No partial writes occur — either the entire transaction succeeds or it is rolled back completely; (c) Application gracefully handles database disconnection by displaying an error page rather than crashing.

NFR-05: Scalability

The database schema shall support growth without requiring structural redesign. The current four-table design (`users`, `faculty`, `feedback`, `comment`) with relationships spanning one-to-many (users→feedback, faculty→feedback via resolved_by_faculty_id), many-to-one (comment→feedback), and self-referencing (comment.parent_id→comment.id for nested replies) can accommodate thousands of users, hundreds of faculty members, and tens of thousands of feedback entries with comment threads without modification. Future additions such as new categories, additional status values, or supplementary tables (e.g., for attachments, notifications) should be possible through ALTER TABLE operations that maintain backward compatibility with existing data.

Acceptance Criteria: Schema supports up to 10,000 users and 50,000 feedback entries without requiring migration scripts or structural changes. New ENUM values can be added to `category` and `status` columns via schema updates.

NFR-06: Maintainability

The codebase shall follow a modular structure that enables future developers to understand, modify, and extend the system with minimal effort. Key maintainability practices implemented include: separation of routes into Flask blueprints by domain (auth, student, admin); distinct model definitions in `models.py`; organized template hierarchy under `templates/` with a base layout template; consistent naming conventions for variables, functions, and files; and comprehensive inline documentation for complex logic blocks.

Acceptance Criteria: A new developer unfamiliar with the project can set up and run the application within 15 minutes by following the README.md installation instructions. Route handlers are organized such that adding a new feature (e.g., "Admin exports feedback to CSV") requires creating or extending only one blueprint file without modifying existing route logic.

NFR-07: Availability

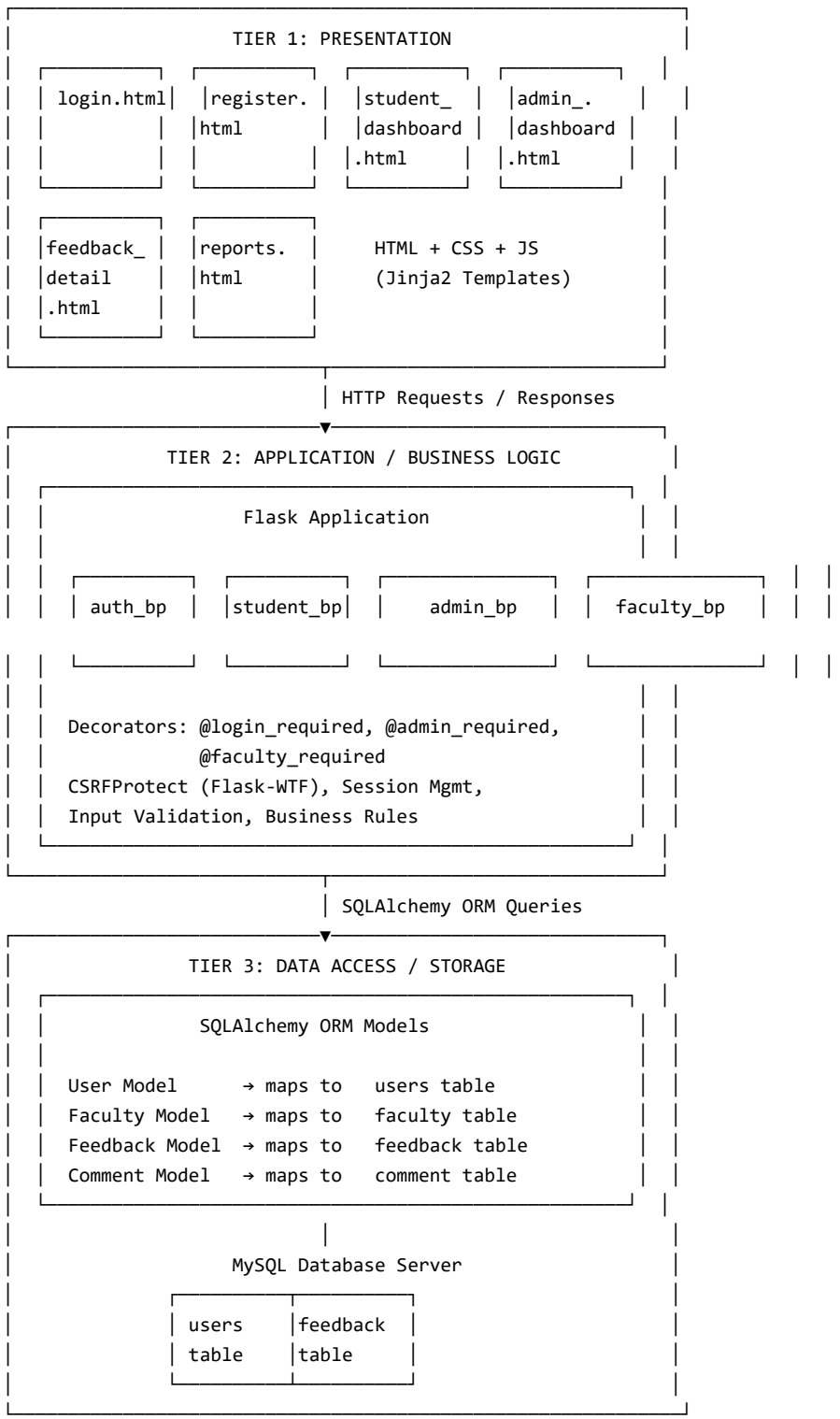
The system shall be capable of running reliably on a local development server for demonstration and deployment purposes during the lab course evaluation period. The Flask development server (`app.py`) shall start successfully with no configuration errors, connect to MySQL without connection failures under normal conditions, and serve requests continuously for at least 8 hours without memory leaks or crashes.

Acceptance Criteria: Application starts without errors on `python app.py`. Server handles concurrent requests from up to 10 users simultaneously without degradation. No unhandled exceptions occur during a sustained usage session of 2+ hours across all routes.

Chapter 9: System Architecture

9.1 Architectural Pattern — Three-Tier Architecture

The Student Feedback Management System follows a three-tier architectural pattern, which separates the application into three logically distinct layers: Presentation Layer (Tier 1), Application/Business Logic Layer (Tier 2), and Data Access/Storage Layer (Tier 3). This separation of concerns is a fundamental principle of software engineering that improves modularity, testability, and maintainability.



9.2 Detailed Layer Descriptions

Presentation Layer (Tier 1)

The presentation layer is responsible for rendering the user interface — everything that users see and interact with in their web browser. It consists of Jinja2 HTML templates located in the `templates/` directory, static assets (CSS stylesheets and JavaScript files) in the `static/` directory, and the Flask templating engine that processes template files with dynamic data passed from route handlers.

Key responsibilities: - Rendering HTML pages using Jinja2 template syntax (`{{ variable }}`, `{% if condition %}`, `{% for item in items %}`) - Displaying flash messages (success, error, warning) to provide user feedback after actions - Form rendering with proper HTTP method attributes and CSRF protection - Client-side validation through JavaScript (required fields, rating range, email format) - Responsive CSS styling using media queries for mobile/tablet/desktop breakpoints

Templates in the project: `base.html` (shared layout with navigation bar and footer), `login.html`, `register.html`, `student_dashboard.html`, `admin_dashboard.html`, `feedback_detail.html`, `reports.html`.

Application Layer (Tier 2)

The application layer contains all business logic — the rules, validations, and workflows that govern how the system behaves. In this project, it is implemented as a Flask web application organized using the Blueprint pattern for modular route separation. The application factory (`create_app()`) centralizes configuration loading, extension initialization (SQLAlchemy), and blueprint registration.

Key responsibilities: - Route handling: Mapping HTTP requests (GET/POST) to specific handler functions - Authentication and authorization: Verifying user credentials via session management; enforcing role-based access through decorators - Input validation: Checking form data for required fields, valid ranges, and format compliance before passing to database operations - Business rule enforcement: Anonymous feedback identity suppression at query layer; status workflow progression rules (Pending → In Progress → Resolved) - Data transformation: Converting raw database records into view-ready structures (e.g., replacing student names with “Anonymous” for anonymous entries)

Blueprints: `blueprints/auth/routes.py` (authentication routes), `blueprints/student/__init__.py` (student dashboard and feedback submission), `blueprints/admin/__init__.py` (admin dashboard, filtering, status updates, verification, faculty management, reports), `blueprints/faculty/routes.py` (faculty dashboard, review, comments, status transitions, resolution verification).

Data Layer (Tier 3)

The data layer manages all interactions with the persistent storage system — in this case, a MySQL database accessed through SQLAlchemy ORM. The ORM provides an abstraction that allows Python code to interact with database tables using object-oriented syntax rather than raw SQL queries, improving code readability and reducing error susceptibility.

Key responsibilities: - Object-relational mapping: Translating between Python objects (`User`, `Feedback`) and database rows in `users` and `feedback` tables - Query construction: Building parameterized SQL queries through SQLAlchemy’s query API to prevent SQL injection - Transaction management: Ensuring data integrity through commit/rollback operations on session boundaries - Relationship navigation: Enabling bidirectional access between related objects (`user.feedback_entries`, `feedback.user`)

Models: `User` model (id, name, email, roll_number, password_hash, role, created_at) — students and admins; `Faculty` model (id, name, email, faculty_id, department, subject_taught, password_hash, role, created_at) — admin-seeded only; `Feedback` model (id, student_id, category, rating, comment, is_anonymous, status, department, subject, semester_year, resolved_by_faculty_id, review_deadline, escalation_deadline, failed_verification_count, created_at, updated_at) with verification methods (`verify_success`, `verify_failure`) and PII-safe serialization (`to_faculty_dict`, `to_admin_dict`); `Comment` model (id, feedback_id, parent_id, author_type, author_id, text, created_at) — self-referencing for nested replies.

9.3 Design Patterns Used

Application Factory Pattern

The `create_app()` function in `app.py` serves as the application factory — a centralized function that creates and configures the Flask application instance. This pattern enables: - Single source of truth for configuration (loaded from `config.py`) - Consistent initialization order (extensions configured before blueprints registered) - Testability through easy creation of separate app instances for testing

Blueprint Pattern

Flask Blueprints provide a mechanism to organize routes into modular, reusable units. Each blueprint encapsulates all routes and handlers related to a specific domain: - `blueprints/auth/routes.py`: All authentication-related routes (register, login, logout) supporting three roles. - `blueprints/student/__init__.py`: Student-facing functionality (submit feedback, view history). - `blueprints/admin/__init__.py`: Admin-only operations (dashboard with filters, status management, verification actions, faculty account management, reports). - `blueprints/faculty/routes.py`: Faculty operations (department-filtered dashboard, comment threads, status transitions, resolution verification, automatic escalation of overdue items).

Decorator Pattern for Access Control

Custom decorators (`@login_required`, `@admin_required`) wrap route handler functions to enforce authentication and authorization checks before the actual business logic executes. This eliminates code duplication across routes and centralizes access control logic.

Chapter 10: Use Case Diagram

10.1 Overview and Description

The use case diagram illustrates the interactions between actors (users) and system functionalities (use cases). There are three primary actors in this system: **Student**, **Admin**, and **Faculty**. A tertiary implicit actor is the **Database**, which stores and retrieves data as needed by the system.

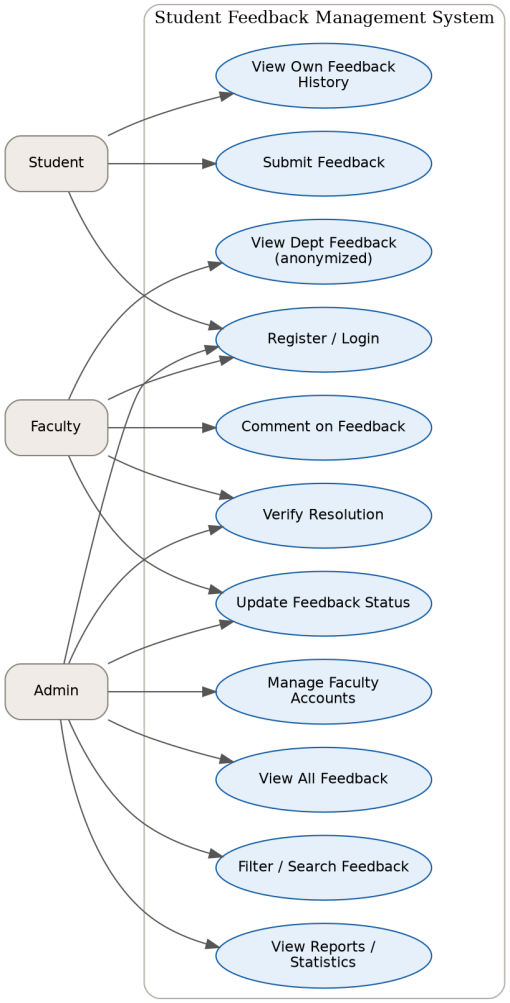


Figure 10.1: Use Case Diagram of the Student Feedback Management System

Actors

1. **Student**: An enrolled student who can register, login, submit feedback, view their own feedback history, read comment threads on their submissions, and logout.
2. **Admin**: An administrative staff member who can login, view all feedback with filtering capabilities, update feedback status, verify resolved items (Verified/Closed or fail), manage faculty accounts, generate reports, and logout.
3. **Faculty**: A teaching staff member who logs in to review department-specific feedback (excluding 5-star ratings), update status through the workflow, post comments on feedback items, verify resolved resolutions, and logout. Faculty accounts are created by admins only — no self-registration.

Use Cases for Student Actor

- **Register**: Create a new account using name, roll number, email, and password. This use case is exclusive to students — admin accounts cannot be self-created.

- **Login**: Authenticate using email or roll number plus password. After login, the system determines the user's role and redirects accordingly.
- **Submit Feedback**: Provide structured feedback including category selection (Food/Faculty/Infrastructure/Events/Other), numerical rating (1–5), text comment, and optional anonymity toggle.
- **View Own Feedback History**: Access a personal page displaying all previously submitted feedback entries with their current status indicators and timestamps.
- **Logout**: Terminate the current session and return to the login page.

Use Cases for Admin Actor

- **Login (Admin)**: Authenticate using admin email and password. This use case is restricted to accounts with role='admin' in the database.
- **View All Feedback**: Access a comprehensive table of all student feedback entries across the system, sorted by most recent first.
- **Filter Feedback**: Apply one or more filter criteria simultaneously — category dropdown, status dropdown, minimum rating input, and date range selection — to narrow the displayed feedback subset.
- **Search Feedback (Keyword)**: Enter a keyword string that is matched against the comment text of all feedback entries to find relevant concerns.
- **View Feedback Detail**: Click on any individual feedback entry to see its complete details including full comment text, submitter information, and metadata.
- **Update Feedback Status**: Change the status of a feedback entry through the defined workflow: Pending → In Progress → Resolved, with automatic timestamp recording.
- **View Reports**: Access an aggregated analytics page displaying average ratings per category, feedback counts by category, and temporal submission trends.
- **Logout**: Terminate the admin session and return to the login page.
- **Manage Faculty Accounts**: List all faculty accounts; create new ones with department, subject_taught assignment, and password.

Use Cases for Faculty Actor

- **Login (Faculty)**: Authenticate using faculty email and password. Restricted to accounts provisioned by admins. Redirects to `/faculty/dashboard` on success.
- **View Department Feedback**: Access a table of feedback matching their department with rating != 5. Items are prioritized by subject match, pinned status, and escalation flags. Student PII is never shown — only class/year derived from roll number.
- **Update Feedback Status**: Change the status of feedback in their department through Pending → In Progress → Resolved, with resolved_by_faculty_id tracked.
- **Post Comments**: Add comments to any feedback item in their department via AJAX (JSON POST with CSRF headers). Supports nested replies.
- **Verify Resolution**: When viewing a Resolved item, mark it Verified/Closed or fail verification (reverts to In Progress, increments failed_verification_count).

Relationships

- **Includes**: "Submit Feedback" includes validation of all input fields (category selection required, rating 1–5 range enforced, comment text validated).
- **Extends**: "Filter Feedback" extends "View All Feedback" — filtering is an optional refinement applied to the base view. Similarly, "Search Feedback" extends "View All Feedback".
- **Generalization**: Student, Admin, and Faculty all share the common use case of "Login", but with different credential sets and post-login redirections.

Chapter 11: Class Diagram

11.1 Overview and Description

The class diagram defines the static structure of the system's object model — four classes are defined as SQLAlchemy ORM models in `models.py`: `User`, `Faculty`, `Feedback`, and `Comment`. These classes encapsulate both data (attributes mapped to database columns) and behavior (methods for validation, relationship navigation, verification lifecycle, and PII-safe serialization).

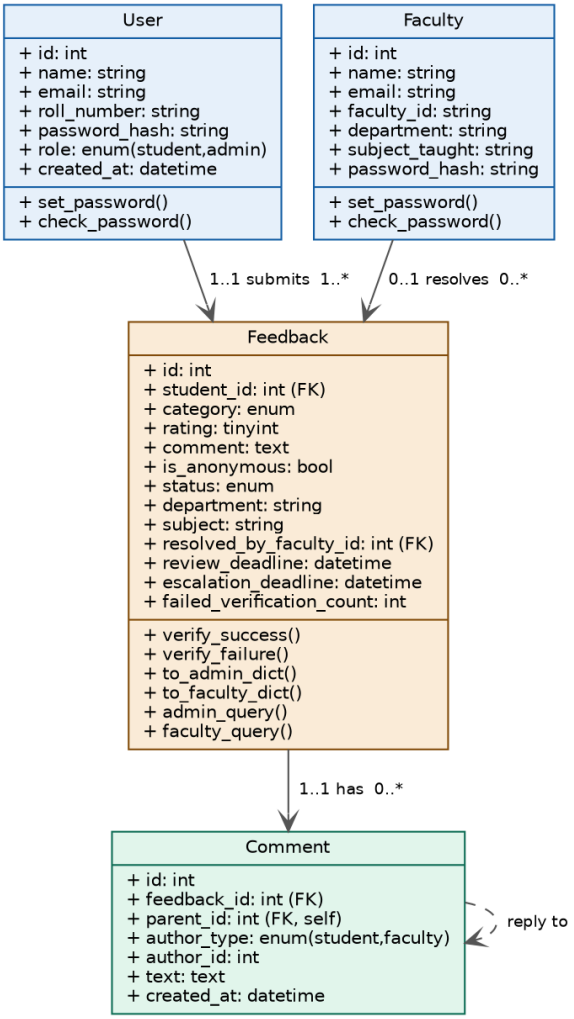


Figure 11.1: Class Diagram (User, Faculty, Feedback, Comment)

Class: User

Attributes: - `id`: Primary key, auto-incremented integer uniquely identifying each user record. - `name`: Full name of the user (student or admin), stored as a string up to 100 characters. - `email`: Email address used for authentication and identification, unique across all users. - `roll_number`: Academic roll number assigned to students; NULL for admin accounts. Unique constraint ensures no duplicate roll numbers among student accounts. - `password_hash`: Bcrypt/PBKDF2-hashed password string generated by werkzeug.security during registration. Never stores plaintext passwords. - `role`: ENUM-like string indicating the user's role — either 'student' or 'admin'. Determines which routes and features are accessible. - `created_at`: Timestamp recording when the account was created, automatically set to the current time on insertion.

Methods: - `validate_password(password)`: Uses werkzeug.security.check_password_hash() to verify whether a provided plaintext password matches the stored hash. Returns boolean. - `get_feedback_entries()`: SQLAlchemy relationship method that returns all Feedback objects associated with this user (one-to-many navigation from User to Feedback). - `__repr__()`: Python string representation for debugging, returning a formatted string like `<User 1: student_name>`.

Class: Faculty

Attributes: - `id`: Primary key, auto-incremented integer. - `name`: Full name of the faculty member. - `email`: Email address used for authentication, unique across all accounts. - `faculty_id`: Unique faculty identifier (e.g., “FAC-001”), set by admin during account creation. - `department`: Department assignment (e.g., “Computer Science”) determining which feedback the faculty can view. - `subject_taught`: Subject(s) taught, used to prioritize matching feedback items on the dashboard. - `password_hash`: Bcrypt/PBKDF2-hashed password generated by werkzeug.security. - `role`: Always ‘faculty’. - `created_at`: Account creation timestamp.

Methods: - `set_password(raw_password)`: Hashes and stores the plaintext password using werkzeug.security.generate_password_hash(). - `check_password(raw_password)`: Verifies a provided password against the stored hash using check_password_hash(). Returns boolean. - `get_resolved_feedback()`: SQLAlchemy relationship returning all Feedback objects where this faculty is the resolver (resolved_by_faculty_id).

Class: Feedback

Attributes: - `id`: Primary key, auto-incremented integer uniquely identifying each feedback entry. - `student_id`: Foreign key referencing the `id` of the User who submitted this feedback. Maintains referential integrity — a feedback record always points to a valid user account. - `category`: ENUM string from predefined set: ‘Food’, ‘Faculty’, ‘Infrastructure’, ‘Events’, ‘Other’. Determines how feedback is categorized and filtered on admin dashboards. - `rating`: TINYINT integer between 1 and 5 (inclusive). Provides quantitative measure of satisfaction for the feedback category. - `comment`: Free-text field allowing students to provide detailed descriptions of their concerns or suggestions. Stored as TEXT type in MySQL to accommodate arbitrarily long comments. - `is_anonymous`: Boolean flag indicating whether the student wishes to remain anonymous. When True, admin-facing views suppress the submitter’s identity at the query layer. - `status`: ENUM string tracking resolution progress through states: ‘Pending’, ‘In Progress’, ‘Resolved’, ‘Pinned’, ‘Verified/Closed’, ‘Verification Failed’. Default value on creation is ‘Pending’. - `department`: VARCHAR(100) NULL — department context for Faculty/Infrastructure feedback categories. - `subject`: VARCHAR(200) NULL — subject context for targeted faculty matching. - `semester_year`: VARCHAR(20) NULL — academic semester/year of the student submitter. - `resolved_by_faculty_id`: Foreign key referencing `faculty.id` (nullable). Records which faculty member resolved this item. - `review_deadline`: DATETIME nullable — 24 hours after submission; used for escalation checks. - `escalation_deadline`: DATETIME nullable — set by `_check_and_escalate()` when review deadline passes without action. - `failed_verification_count`: TINYINT DEFAULT 0 — tracks how many times verification has failed. - `created_at`: Timestamp of when feedback was originally submitted, set automatically on insertion. - `updated_at`: Timestamp of the most recent status change, updated automatically whenever admin modifies the status field.

Methods: - `verify_success()`: Transitions status from ‘Resolved’ → ‘Verified/Closed’. Raises ValueError if current status is not ‘Resolved’. Called by both admin and faculty verification actions. - `verify_failure()`: Transitions status from ‘Resolved’ → ‘In Progress’, increments failed_verification_count by 1. Raises ValueError if current status is not ‘Resolved’. Implements the failed-verification loop that can trigger escalation when count reaches 3. - `to_faculty_dict(faculty_subject)`: Returns an anonymized dict safe for faculty views — includes class/year derived from roll_number (never name/email/roll), subject match flag, escalation flags. Never joins users table. - `to_admin_dict()`: Returns a dict for admin views — attaches student PII only when is_anonymous=False; otherwise returns “Anonymous” for all identity fields. - `__repr__()`: Python string representation returning formatted string like `<Feedback 1: Food(4/5)>`.

Class: Comment

Attributes: - `id`: Primary key, auto-incremented integer uniquely identifying each comment record. - `feedback_id`: Foreign key referencing the `id` of the Feedback item this comment belongs to. Ensures every comment is tied to a valid feedback entry. - `parent_id`: Integer foreign key referencing `comment.id` (nullable). Enables self-referencing nested replies — a parent comment can have many child reply comments. - `author_type`: ENUM string (‘faculty’ or ‘student’) indicating whether the commenter is a faculty member or the submitting student. - `author_id`: Integer ID of the author, corresponding to either a `faculty.id` (for faculty) or `users.id` (for students). - `text`: Free-text content of the comment. Stored as TEXT type in MySQL to support arbitrarily long messages. - `created_at`: Timestamp of when the comment was created, automatically set on insertion.

Methods: - `__repr__()`: Python string representation returning formatted string like `<Comment N: Feedback M>`.

Class Relationships

Relationship	Type	Description
User → Feedback	One-to-Many	A single user can submit multiple feedback entries. Implemented via `db.relationship('Feedback', backref='author')` on the User model and `db.ForeignKey('users.id')` on the Feedback model's `student_id`.
Faculty → Feedback (resolved)	One-to-Many	A faculty member can resolve multiple feedback items. Implemented via `db.relationship('Feedback', backref='resolver', foreign_keys='Feedback.resolved_by_faculty_id')` and `db.ForeignKey('faculty.id')` on the Feedback model's `resolved_by_faculty_id`.
Comment → Feedback	Many-to-One	Each comment belongs to exactly one feedback item. FK: `feedback_id` → `feedback.id`.
Comment → Comment (self-ref)	One-to-Many (nested replies)	A parent comment can have many child reply comments. Self-referencing FK: `parent_id` → `comment.id`. Enables Reddit-style threaded discussions.
User/Faculty.password_hash	Encapsulation	Password hashing is encapsulated within both User and Faculty classes through werkzeug.security methods — external code never accesses or manipulates raw passwords.

Cardinality Notation

- **User** ———(1)———◆———(Many)——— **Feedback**
- The diamond (◆) indicates a strong relationship where Feedback cannot exist without an associated User (referential integrity enforced by foreign key constraint).
- Each User can have zero or more associated Feedback entries (a newly registered student with no submitted feedback still has a valid User record with an empty `feedback_entries` collection).

Chapter 12: Sequence Diagrams

12.1 Sequence for “Submit Feedback” (Student)

This sequence diagram describes the interaction flow when an authenticated student submits a new feedback entry through `blueprints/student/__init__.py`. The interaction spans multiple system components from the user’s browser to the database and back.

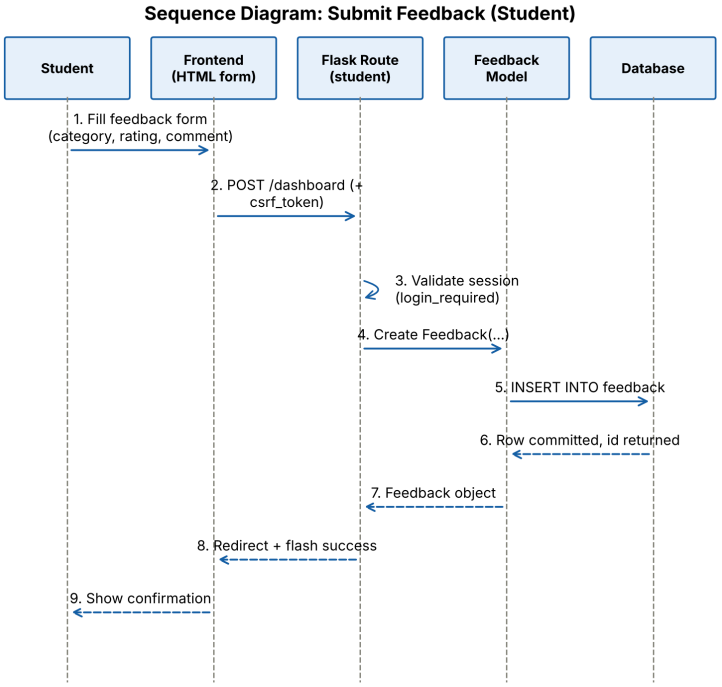


Figure 12.1: Sequence Diagram - Submit Feedback

Sequence Steps:

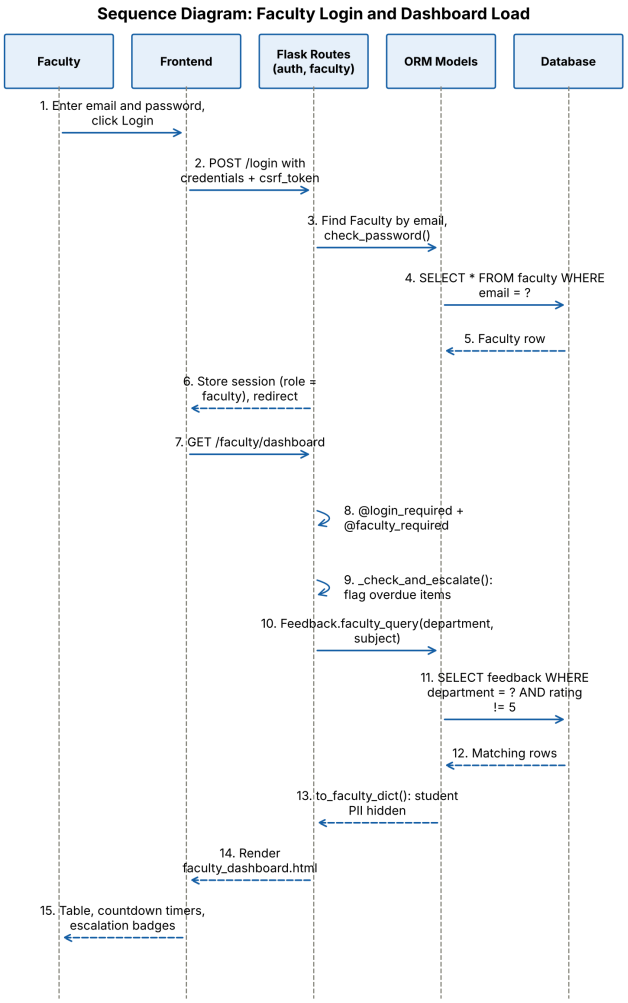
1. **Student → Frontend**: Click “Submit Feedback” button on `student_dashboard.html` form.
2. **Frontend → Student**: Client-side validation (required fields, rating 1-5 range).
3. **Frontend → Flask Route Handler**: HTTP POST to `/dashboard` with form data (`category`, `rating`, `comment`, anonymous checkbox, and conditional fields: `department`, `subject`, `semester_year`).
4. **Flask Route Handler → Flask Route Handler**: Verify session[‘user_id’] exists (enforced by `@login_required` decorator).
5. **Flask Route Handler → ORM Model**: Create Feedback object with `student_id=session['user_id']`, parsed form fields, and `review_deadline=datetime.now(UTC) + 24h`.
6. **ORM Model → Database Session**: `db.session.add(feedback_obj)`.
7. **Database Session → MySQL/SQLite**: INSERT INTO feedback VALUES (…).
8. **MySQL/SQLite → Database Session**: Confirmation of insert (row ID assigned).
9. **Database Session → Flask Route Handler**: `db.session.commit()` returns success.
10. **Flask Route Handler → Frontend**: Redirect to `/dashboard` with flash message “Feedback submitted successfully!”.
11. **Frontend → Student**: Display updated dashboard showing new entry in history table.

Alternative Flow (Validation Failure):

If any validation fails (e.g., rating outside 1–5, missing required fields), the Flask route handler returns an error response with appropriate flash messages, and the frontend displays the errors without committing to the database.

12.2 Sequence for “Faculty Login and Dashboard Load”

This sequence shows multi-role authentication followed by the faculty dashboard loading with department filtering, automatic escalation



check, and PII-safe serialization.

Figure 12.2: Sequence Diagram - Faculty Login and Dashboard Load

Participants:

1. **Faculty** — End-user via web browser
2. **Frontend (login.html)** — Multi-role login form with CSRF token
3. **Flask Route Handler** (`blueprints/auth/routes.py::login`) — Tries User first, then Faculty on mismatch
4. **ORM Model** (`Faculty` class in `models.py`) — Department/subject lookup
5. **Flask Route Handler** (`blueprints/faculty/routes.py::dashboard`) — Department filtering, escalation check
6. **ORM Model** (`Feedback.faculty_query()`, `to_faculty_dict()`) — PII-safe query and serialization
7. **Database Session / MySQL-SQLite** — Persistent storage

Sequence Steps:

1. **Faculty enters email and password** on `/login`; clicks Login (CSRF token included in form).
2. **Frontend POSTs to** `/login` (`blueprints/auth/routes.py::login`) with credentials and `csrf_token`.
3. **auth/login tries User table first**: queries `users WHERE email = ?` via `User.query.filter_by(email=email).first()`; if match, calls `user.check_password(password)` (uses `werkzeug.check_password_hash`); sets `session['user_id']`, `session['role']=user.role`; redirects based on role.
4. **If User didn’t match, tries Faculty table**: queries `faculty WHERE email = ?` via `Faculty.query.filter_by(email=email).first()`; if match, calls `faculty.check_password(password)`; sets

`session['faculty_id']`, `session['role']='faculty'`, `session['name']=faculty.name`; redirects to `/faculty/dashboard`.

5. **If neither table matches**: flashes "Invalid email or password." (non-specific, prevents enumeration) and stays on login page.
6. **GET** `/faculty/dashboard` — frontend navigates to faculty dashboard after successful authentication.
7. **Route handler** `dashboard()` (`blueprints/faculty/routes.py::dashboard`) is protected by `@login_required` (checks `'user_id' or 'faculty_id' in session`) and `@faculty_required` (checks `session['role']=='faculty'`).
8. `_check_and_escalate()` **runs**: queries feedback where `department == faculty.department`, `review_deadline` has passed, `escalation_deadline IS NULL`, status not `'Verified/Closed'` or `'Verification Failed'`; sets `status='Pinned'` and `escalation_deadline=now+3 days` for each overdue item; commits changes.
9. `Feedback.faculty_query()` **called** with `department=faculty.department, subject=faculty.subject_taught`; filters to exclude 5-star ratings (`rating != 5`).
10. **SQL SELECT** executes: `SELECT * FROM feedback WHERE department=? AND rating!=5`, ordered by `subject_match DESC, created_at DESC` (or pinned first if no subject filter).
11. **Matching rows returned** from the database query.
12. `to_faculty_dict()` **serializes each row**: hides student PII — only class/year derived from roll_number is shown; includes id, category, rating, comment, status, subject_match flag, and is_escalated flag.
13. `faculty_dashboard.html` **rendered** with `feedback_list`, `pending_count`, and `escalated_count` passed as template context variables.
14. **Dashboard displayed**: stats cards (total feedback, pending review count, escalated item count), department-filtered feedback table with countdown timers for deadlines, and escalation badges for overdue items.

12.3 Post Comment (Faculty, AJAX)

This sequence describes how a matching faculty member posts a comment on a feedback item via an AJAX JSON POST request from the frontend.

1. **Faculty/Student → Frontend**: Type comment in textarea; click Post button.
2. **Frontend → JavaScript**: Read CSRF token from `<meta name="csrf-token">` element.
3. **Frontend → Flask Route Handler**: HTTP POST to `/faculty/feedback/<id>/comment` with JSON body `{"text": "..."}` and X-CSRFToken header.
4. **Flask Route Handler → Flask Route Handler**: Verify session['role']=='faculty' AND feedback department matches faculty department.
5. **Flask Route Handler → ORM Model (Comment)**: Create new Comment(feedback_id=id, author_type='faculty', author_id=faculty.id, text=text).
6. **ORM Model → Database Session**: `db.session.add(comment)`; `db.session.commit()`.
7. **Database Session → MySQL/SQLite**: INSERT INTO comment VALUES (…).
8. **alt JSON request (AJAX)**: Flask Route Handler → Frontend: 201 `{"id": N, "text": "...", "created_at": "..."}`; Frontend → JavaScript: Append comment div to `.comment-thread` element.
9. **else Form POST (non-AJAX)**: Flask Route Handler → Frontend: Flash message + redirect.

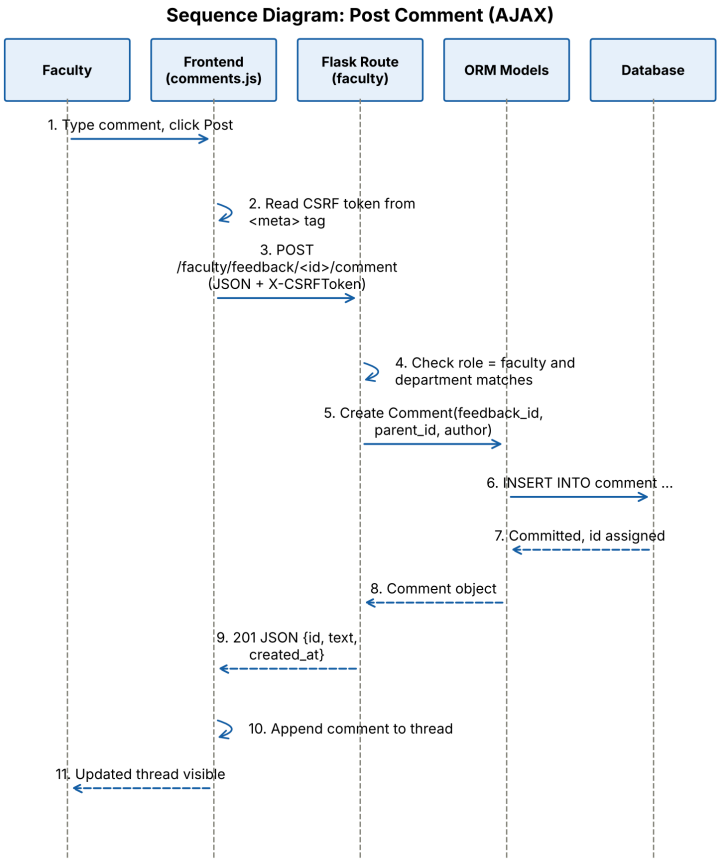


Figure 12.3: Sequence Diagram - Post Comment

Figure 12.3: Sequence Diagram - Post Comment

Participants:

1. **Faculty** — End-user typing a comment in the discussion thread
2. **Frontend (faculty_feedback_detail.html)** — Comment form with inline JavaScript using comments.js / submitComment()
3. **Flask Route Handler** (`blueprints/faculty/routes.py::post_comment`) — JSON POST handler for comment creation
4. **ORM Model** (`Comment` class in `models.py`) — Creates new Comment record
5. **Database Session / MySQL-SQLite** — Persistent storage

12.4 Verify Resolved Feedback (Faculty or Admin)

This sequence describes the verification workflow when a faculty member or admin verifies a resolved feedback item. Both verify methods (`verify_success()` and `verify_failure()`) raise ValueError unless status == ‘Resolved’.

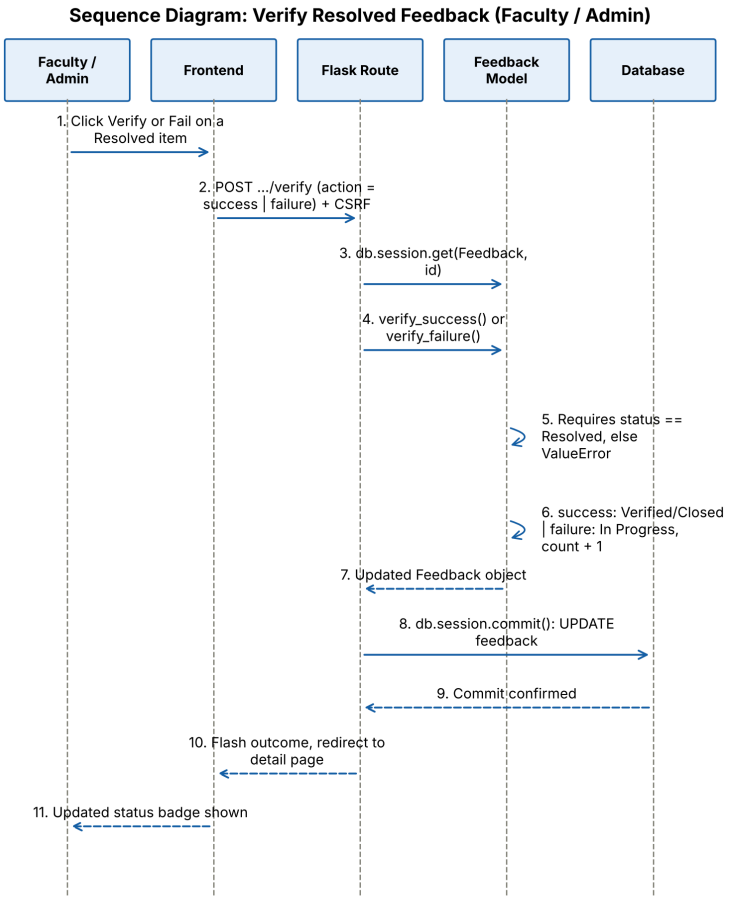


Figure 12.4: Sequence Diagram - Verify Resolved Feedback

Participants:

1. **Admin or Faculty** — End-user clicking verification buttons on a Resolved feedback detail page
2. **Frontend (admin_feedback_detail.html / faculty_feedback_detail.html)** — Verification action forms with CSRF tokens
3. **Flask Route Handler** (`blueprints/admin/__init__.py::verify_success/verify_failure` OR `blueprints/faculty/routes.py::verify_feedback`) — POST handlers for verification actions
4. **ORM Model** (`Feedback.verify_success()`, `Feedback.verify_failure()` in `models.py`) — Verification lifecycle methods
5. **Database Session / MySQL-SQLite** — Persistent storage

Sequence Steps:

1. **Admin/Faculty → Frontend**: Click "✓ Verify" or "✗ Fail" on a Resolved feedback item.
2. **Frontend → Flask Route Handler**: HTTP POST to `/admin/feedback/<id>/verify-success` OR `/faculty/feedback/<id>/verify` with `action='success'/'failure'` and csrf_token.
3. **Flask Route Handler → ORM Model (Feedback)**: Load feedback (`fb = db.session.get(Feedback, id)`); verify status == 'Resolved' before calling verification methods.
4. **alt Action = 'success'**: ORM Model calls `fb.verify_success()` → sets status = 'Verified/Closed'.
5. **else Action = 'failure'**: ORM Model calls `fb.verify_failure()` → increments `failed_verification_count` by 1, sets status = 'In Progress'; if count >= 3, item is flagged as escalated.
6. **ORM Model → Database Session**: `db.session.commit()`.
7. **Database Session → MySQL/SQLite**: UPDATE feedback SET status=?, failed_verification_count=? WHERE id=?.
8. **Flask Route Handler → Frontend**: Flash message confirming outcome.
9. **Frontend → Admin/Faculty**: Redirect back to detail page; updated status badge shown.

12.5 Admin Update Feedback Status

This sequence describes how an admin updates the status of a feedback item through the Pending → In Progress → Resolved workflow.

Participants:

1. **Admin** — End-user selecting new status from dropdown on feedback detail page
2. **Frontend (admin_feedback_detail.html)** — Status update form with CSRF token
3. **Flask Route Handler** (`blueprints/admin/__init__.py::update_status`) — POST handler for status updates
4. **ORM Model** (`Feedback` class in `models.py`) — Updates status field and timestamp
5. **Database Session / MySQL-SQLite** — Persistent storage

Sequence Steps:

```
Admin → Frontend: Click "View" on a feedback entry in admin dashboard table
Frontend → Flask Route Handler (admin/__init__.py::feedback_detail): HTTP GET to /admin/feedback/<id>
Flask Route Handler → ORM Model (Feedback): fb = db.session.get(Feedback, id) or get_or_404(id)
ORM Model → MySQL Database: SELECT * FROM feedback WHERE id = ?
MySQL Database → ORM Model: Return Feedback record with current status and details

ORM Model → Flask Route Handler: Render admin_feedback_detail.html with full feedback data
Frontend → Admin: Display detail page with full comment, metadata, and status update form

Admin → Frontend: Select new status (Pending/In Progress/Resolved) from dropdown; click "Update"
Frontend → Flask Route Handler: HTTP POST to /admin/feedback/<id>/status with new_status value + csrf_

Flask Route Handler → Flask Route Handler: Verify session role == 'admin' (@admin_required decorator)
Flask Route Handler → ORM Model: fb.status = new_status; db.session.commit()
ORM Model → MySQL Database: UPDATE feedback SET status=?, updated_at=NOW() WHERE id=?
MySQL Database → ORM Model: Confirmation of update

Flask Route Handler → Frontend: Flash message "Status updated to [new_status]"
Frontend → Admin: Redirect back to admin dashboard; table now shows updated status badge
```

Chapter 13: Activity Diagrams

13.1 Feedback Status Lifecycle (Activity Flow)

This activity diagram describes the state transitions that a feedback entry undergoes from initial submission through eventual resolution or closure. Unlike a simple linear workflow, this lifecycle involves multiple actors (Faculty and Admin), verification decision points with a failed-verification loop back to In Progress, and escalation checks when deadlines pass.

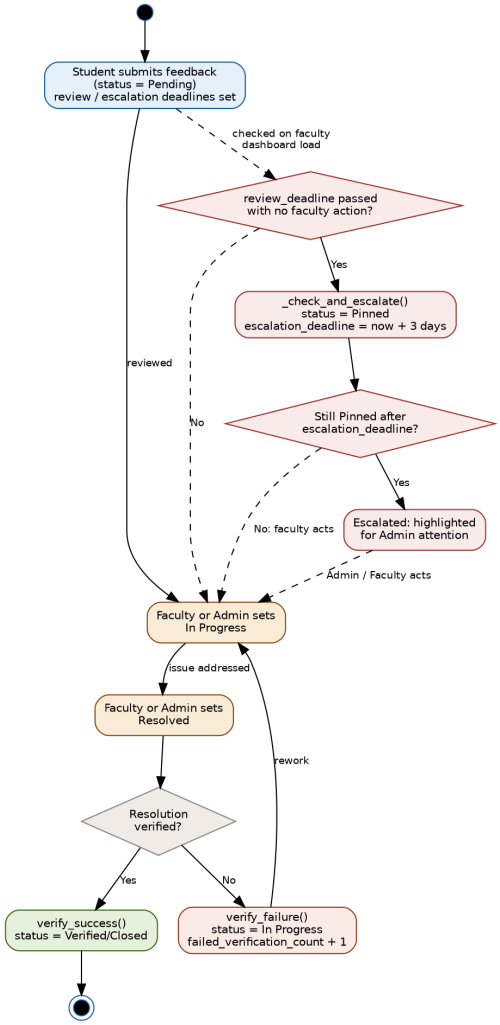


Figure 13.1: Activity Diagram - Feedback Status Lifecycle

Process Flow:

1. **Student submits feedback** → status set to `Pending`; `review_deadline` = submission time + 24 hours.
2. **Faculty or Admin reviews item** on their dashboard (department-filtered for faculty, all items for admin).
3. **Status transitions to In Progress** — Faculty or Admin marks the item as being actively addressed; `resolved_by_faculty_id` recorded if a faculty member does it.
4. **Issue is worked on** — may involve coordination between stakeholders; status remains In Progress.
5. **Faculty/Admin resolves issue** → status set to `Resolved`; triggers verification requirement.
6. **Verification step**: An Admin or Faculty member evaluates whether the resolution was adequate.
    - If adequate: calls `verify_success()` → status transitions to `Verified/Closed` (terminal state).
    - If inadequate: calls `verify_failure()` → status reverts to `In Progress`, `failed_verification_count` incremented by 1.
7. **Failed-verification loop**: Each verification failure increments the counter; when `failed_verification_count >= 3`, the item is flagged as escalated on next dashboard load.

8. **Escalation check** (`_check_and_escalate()`): Runs on every Faculty/Admin dashboard load — finds items where `review_deadline` has passed without status transition, sets status to `Pinned` and `escalation_deadline` = now + 3 days.
9. **Verified/Closed**: Terminal state — no further transitions; comments are disabled on the detail page.

State Definitions:

- **Pending**: The feedback has been submitted by a student but has not yet been reviewed. This is the initial and default state for all newly created feedback records. A `review_deadline` (24 hours from submission) is set automatically.
- **In Progress**: An administrator or faculty member has begun reviewing or acting on the item. The issue is actively being addressed. Items can return to this state via verification failure (`verify_failure()` increments `failed_verification_count`).
- **Resolved**: A faculty or admin marks the issue as addressed. This is not a terminal state — it requires verification before the item can be closed. When marked Resolved by a faculty member, `resolved_by_faculty_id` is recorded.
- **Pinned**: An overdue item that has passed its review deadline without status transition. Automatically set by `_check_and_escalate()` when the faculty dashboard loads. Escalated items are highlighted with red background and warning badges on both admin and faculty dashboards.
- **Verified/Closed**: The terminal state — an admin or faculty member verified via `verify_success()` that the resolution was adequate. No further status transitions occur from this state; comments are disabled on the detail page.
- **Verification Failed**: This is not a separate status value in the code. When verification fails, `verify_failure()` sets status back to 'In Progress' and increments `failed_verification_count`. The counter is displayed on dashboards as a metric (items with count >= 3 are flagged as escalated).

Transitions:

From State	To State	Triggering Action	Actor
Pending	In Progress	Admin or faculty reviews item and begins action	Faculty or Admin
In Progress	Resolved	Issue addressed; marked by admin or faculty	Faculty or Admin
Resolved	Verified/Closed	`verify_success()` called — resolution verified adequate	Faculty or Admin
Resolved	In Progress	`verify_failure()` called — resolution inadequate, counter incremented	Faculty or Admin
Pending/In Progress	Pinned	`_check_and_escalate()` detects overdue item (past review_deadline)	Automatic
Any escalated state	Flagged as escalated	`failed_verification_count >= 3` or past escalation_deadline	Automatic

Failed Verification Loop:

When a resolved item is verified as failed, the workflow loops back to In Progress with `failed_verification_count` incremented. Each subsequent verification failure increments the counter further. When the counter reaches 3 (or more), `_check_and_escalate()` will flag the item as escalated on the next faculty dashboard load, setting it to Pinned status if not already — creating a feedback loop that ensures problematic items receive administrative attention.|

13.2 Student Registration Activity Flow

This activity diagram describes the flow when a new student registers for an account through the `/register` endpoint in `blueprints/auth/routes.py`. The process includes client-side validation, uniqueness checks on both roll number and email, password hashing, and database insertion.

1. **User navigates to** `/register` page (`register.html`).
2. **User fills registration form**: enters name, roll number, email address, and password (minimum 6 characters).
3. **Client-side validation** (JavaScript): checks required fields are filled, email format is valid, and password meets minimum length requirement.
4. **If validation fails**: display inline error messages; user stays on the registration page to correct inputs.
5. **If validation passes**: form submits via HTTP POST to `/register` with all fields plus `csrf_token` included in the form data.
6. **Server receives POST request** and validates CSRF token (Flask-WTF).
7. **Uniqueness check**: Query `users` table for existing record with matching `roll_number` AND matching `email`. If either already exists, flash an error message and return to registration form.
8. **Hash password**: Call `generate_password_hash(password)` via werkzeug.security (PBKDF2-SHA256).

9. **Create User object** with `role='student'` and insert into database: `INSERT INTO users (name, roll_number, email, password_hash, role) VALUES (…)`.
10. **Commit transaction**: `db.session.commit()`;
11. **Flash success message** "Registration successful! Please login." and redirect to `/login`.

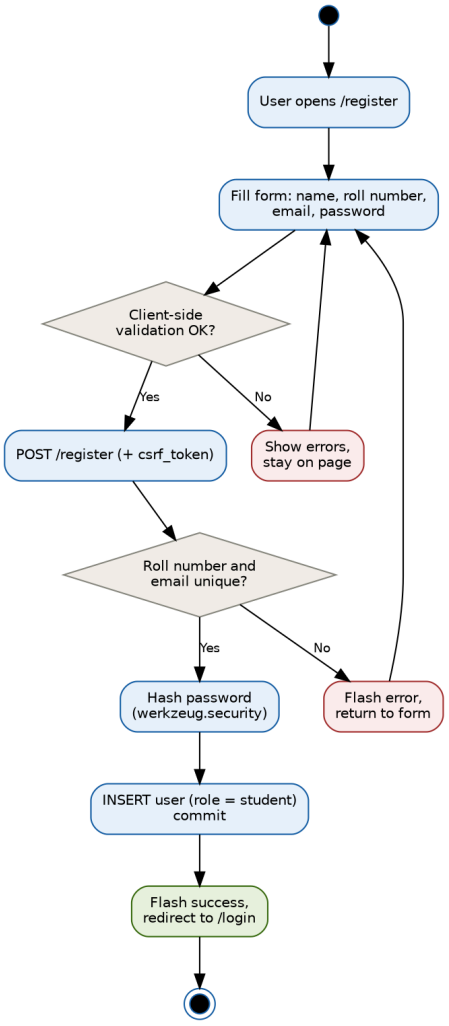


Figure 13.2: Activity Diagram - Student Registration

Figure 13.2: Activity Diagram - Student Registration

13.3 Admin Filtering Activity Flow

This activity diagram describes how an administrator applies multi-criteria filters on the admin dashboard to narrow down feedback entries. The filtering logic is implemented in `blueprints/admin/__init__.py` and supports five independent filter dimensions.

1. **Admin views** `/admin/dashboard` — default display shows all feedback entries (no filter applied).
2. **Admin selects filter criteria** from the filter bar:
   - **Category**: Dropdown selection (Food / Faculty / Infrastructure / Events / Other) — filters on `feedback.category`.
   - **Status**: Dropdown selection (Pending / In Progress / Resolved) — filters on `feedback.status`.
   - **Rating**: Exact numeric match (1–5) — filters on `feedback.rating`.
   - **Date range**: Two date pickers (`date_from`, `date_to`) — filters on `feedback.created_at BETWEEN ? AND ?`.
   - **Keyword search**: Text input — performs `LIKE '%keyword%'` search across the `comment` column (case-insensitive).
3. **Admin clicks Apply Filters** or triggers filter via AJAX as criteria change.
4. **Frontend sends request** with selected filter parameters to `/admin/dashboard?category=...&status=...&rating=...&date_from=...&date_to=...&keyword=...`.
5. **Flask route handler builds query**: Starts with base `Feedback.query`, applies each non-empty filter as a `.filter()` clause using SQLAlchemy's chained query builder.

6. **Execute query** and return filtered result set to frontend.
7. **Frontend re-renders** the feedback table with only matching entries; pagination controls update accordingly.
8. **Admin can clear all filters** — resets form fields and reloads unfiltered view.

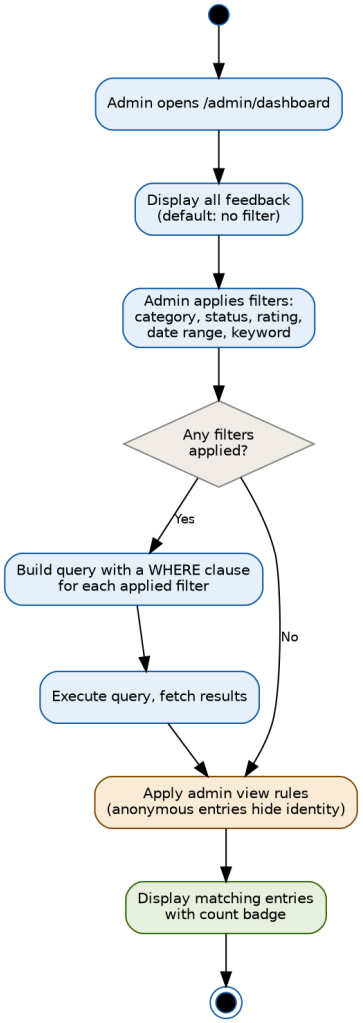


Figure 13.3: Activity Diagram - Admin Dashboard Filtering

Figure 13.3: Activity Diagram - Admin Dashboard Filtering

Chapter 14: ER Diagram

14.1 Overview and Description

The Entity-Relationship (ER) diagram models the logical structure of the database, showing four core entities (tables), their attributes (columns), primary keys, foreign keys, and relationships between entities. The database consists of four entities with well-defined referential integrity constraints.

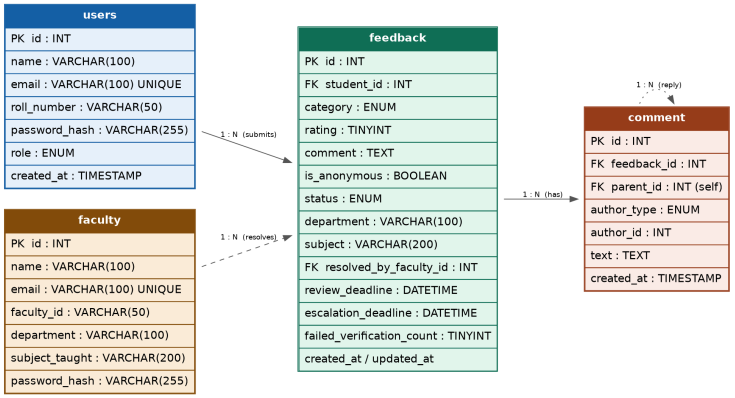


Figure 14.1: Entity-Relationship Diagram

Entity: User

Attribute	Type	Constraints	Description
`id`	INT	PK, AUTO_INCREMENT	Unique identifier for each user account
`name`	VARCHAR(100)	NOT NULL	Full name of the user (student or admin)
`email`	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for authentication and identification
`roll_number`	VARCHAR(50)	UNIQUE, NULL	Academic roll number assigned to students; NULL for admin accounts
`password_hash`	VARCHAR(255)	NOT NULL	Bcrypt/PBKDF2-hashed password generated by werkzeug.security during registration
`role`	ENUM(‘student’,‘admin’)	NOT NULL, DEFAULT ‘student’	User role — determines which routes and features are accessible
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp, automatically set on insertion

Entity: Faculty

Attribute	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for each faculty account
name	VARCHAR(100)	NOT NULL	Full name of the faculty member
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for authentication
faculty_id	VARCHAR(50)	UNIQUE, NOT NULL	Unique faculty identifier (e.g., “FAC-001”), set by admin during account creation
department	VARCHAR(100)	NOT NULL	Department assignment determining which feedback the faculty can view
subject_taught	VARCHAR(200)	NOT NULL	Subject(s) taught, used to prioritize matching feedback items on the dashboard
password_hash	VARCHAR(255)	NOT NULL	Bcrypt/PBKDF2-hashed password generated by werkzeug.security
role	ENUM(‘faculty’)	NOT NULL, DEFAULT ‘faculty’	Always ‘faculty’; faculty accounts are created by admins only — no self-registration
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Entity: Feedback

Attribute	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for each feedback entry
student_id	INT	FK → users.id, NOT NULL	References the student who submitted this feedback; ON DELETE RESTRICT prevents orphaned entries
category	ENUM('Food','Faculty','Infrastructure','Events','Other')	NOT NULL	Feedback category used for filtering and reporting
rating	TINYINT	NOT NULL, CHECK 1-5	Numerical satisfaction rating from 1 (worst) to 5 (best)
comment	TEXT	NOT NULL	Free-text description of the student's concern or suggestion
is_anonymous	BOOLEAN	DEFAULT FALSE	When TRUE, admin-facing queries suppress submitter identity at query level
status	ENUM('Pending','In Progress','Resolved','Pinned','Verified/Closed','Verification Failed')	NOT NULL, DEFAULT 'Pending'	Resolution workflow state
department	VARCHAR(100)	NULL	Department context for Faculty/Infrastructure categories
subject	VARCHAR(200)	NULL	Subject context for targeted faculty matching
semester_year	VARCHAR(20)	NULL	Academic semester/year of the submitting student
resolved_by_faculty_id	INT	FK → faculty.id, NULL	Records which faculty member resolved this item
review_deadline	DATETIME	NULL	24 hours after submission; used for escalation checks
escalation_deadline	DATETIME	NULL	Set by `_check_and_escalate()` when review deadline passes without action
failed_verification_count	TINYINT	DEFAULT 0	Tracks how many times verification has failed; >= 3 triggers escalation flag
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Feedback submission timestamp
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP, ON UPDATE	Most recent status change timestamp

Entity: Comment

Attribute	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for each comment record
feedback_id	INT	FK → feedback.id, NOT NULL	References the feedback item this comment belongs to
parent_id	INT	FK → comment.id, NULL	Self-referencing foreign key enabling nested replies; NULL for top-level comments
author_type	ENUM('faculty','student')	NOT NULL	Indicates whether the commenter is a faculty member or the submitting student
author_id	INT	NOT NULL	ID of the author — corresponds to `faculty.id` (for faculty) or `users.id` (for students)
text	TEXT	NOT NULL	Free-text content of the comment; supports arbitrarily long messages
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Comment creation timestamp

Relationships

Relationship	Type	Description
User → Feedback	One-to-Many (1:N)	Each User can submit zero or more Feedback entries. FK: `feedback.student_id` → `users.id`. ON DELETE RESTRICT prevents orphaned feedback.
Faculty → Feedback (resolved)	One-to-Many (1:N)	A faculty member can resolve multiple feedback items. FK: `feedback.resolved_by_faculty_id` → `faculty.id`. NULL allows unassigned resolved items.
Comment → Feedback	Many-to-One (N:1)	Each comment belongs to exactly one feedback item. FK: `comment.feedback_id` → `feedback.id`.
Comment → Comment (self-ref)	One-to-Many (nested replies)	A parent comment can have many child reply comments. Self-referencing FK: `comment.parent_id` → `comment.id`. Enables Reddit-style threaded discussions.

Cardinality Interpretation:

- **User → Feedback (1:N)**: A single user record is referenced by zero, one, or many feedback records. Each individual feedback entry points back to exactly one user via `student_id`.
- **Faculty → Feedback resolved (1:N)**: A faculty member can resolve multiple items; each resolved item references at most one resolver.
- **Comment → Feedback (N:1)**: Many comments belong to one feedback item.
- **Comment → Comment self-ref (1:N)**: One parent comment can have many nested replies, enabling threaded discussions.

Chapter 15: Database Design

15.1 Schema Overview

The database consists of four tables (`users`, `faculty`, `feedback`, `comment`) connected by foreign key relationships. The schema was designed following normalization principles (up to Third Normal Form) to minimize data redundancy while maintaining query efficiency for the application's access patterns. All tables use MySQL-compatible types with SQLAlchemy ORM abstractions that work transparently on SQLite for testing.

15.2 Table: users

Column	Data Type	Constraints	Description
`id`	INT	PK, AUTO_INCREMENT	Unique identifier for each user
`name`	VARCHAR(100)	NOT NULL	Full name of the user
`email`	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for authentication
`roll_number`	VARCHAR(50)	UNIQUE, NULL	Academic roll number (NULL for admin accounts)
`password_hash`	VARCHAR(255)	NOT NULL	Hashed password (PBKDF2-SHA256 with salt)
`role`	ENUM	NOT NULL, DEFAULT 'student'	User role: 'student' or 'admin'
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Indexes: - PRIMARY KEY on `id` - UNIQUE INDEX on `email` - UNIQUE INDEX on `roll_number`

15.3 Table: faculty

Column	Data Type	Constraints	Description
`id`	INT	PK, AUTO_INCREMENT	Unique identifier for each faculty account
`name`	VARCHAR(100)	NOT NULL	Full name of the faculty member
`email`	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for authentication
`faculty_id`	VARCHAR(50)	UNIQUE, NOT NULL	Unique faculty identifier (e.g., "FAC-001")
`department`	VARCHAR(100)	NOT NULL	Department assignment (filters feedback visibility)
`subject_taught`	VARCHAR(200)	NOT NULL	Subject(s) taught, used for priority matching on dashboard
`password_hash`	VARCHAR(255)	NOT NULL	Hashed password (PBKDF2-SHA256 with salt)
`role`	ENUM	NOT NULL, DEFAULT 'faculty'	Always 'faculty'
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Indexes: - PRIMARY KEY on `id` - UNIQUE INDEX on `email` - UNIQUE INDEX on `faculty_id`

15.4 Table: feedback

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for each feedback entry
student_id	INT	FK → users.id, NOT NULL	Reference to the submitting user
category	ENUM	NOT NULL	Feedback category: Food/Faculty/Infrastructure/Events/Other
rating	TINYINT	CHECK (1 ≤ rating ≤ 5)	Numerical satisfaction rating
comment	TEXT	NOT NULL	Free-text feedback description
is_anonymous	BOOLEAN	DEFAULT FALSE	Whether student identity is hidden from admins
status	ENUM	NOT NULL, DEFAULT 'Pending'	Resolution status: Pending/In Progress/Resolved/Pinned/Verified/Closed/Verification Failed
department	VARCHAR(100)	NULL	Department for routing to faculty
subject	VARCHAR(200)	NULL	Subject for matching with faculty subjects
semester_year	VARCHAR(20)	NULL	Semester/year of the student
resolved_by_faculty_id	INT	FK → faculty.id, NULL	Faculty who resolved this item
review_deadline	TIMESTAMP	NULL	Deadline for initial review (24h from submission)
escalation_deadline	TIMESTAMP	NULL	Escalation deadline (3 days from detection of overdue)
failed_verification_count	TINYINT	NOT NULL, DEFAULT 0	Counter incremented on failed verifications
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Feedback submission timestamp
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last status change timestamp

Indexes: - PRIMARY KEY on `id` - FOREIGN KEY on `student_id` referencing `users(id)` with RESTRICT delete behavior - FOREIGN KEY on `resolved_by_faculty_id` referencing `faculty(id)` - INDEX on `category` (for filtering queries) - INDEX on `status` (for filtering queries) - INDEX on `created_at` (for sorting and date-range queries)

15.5 Table: comment

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for each comment
feedback_id	INT	FK → feedback.id, NOT NULL	Reference to the parent feedback item
parent_id	INT	FK → comment.id, NULL	Self-referencing FK for nested replies
author_type	ENUM	NOT NULL	'faculty' or 'student'
author_id	INT	NOT NULL	ID of the author (matches appropriate table)
text	TEXT	NOT NULL	Comment text content
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Comment creation timestamp

Indexes: - PRIMARY KEY on `id` - FOREIGN KEY on `feedback_id` referencing `feedback(id)` with RESTRICT delete behavior - FOREIGN KEY on `parent_id` referencing `comment(id)` (self-reference, ON DELETE SET NULL) - INDEX on `feedback_id` (for loading comment threads per feedback item)

15.6 Status Enum Details

The `feedback.status` column supports the following values in order of the workflow lifecycle:

Value	Description
Pending	Default state when a student submits feedback; visible on admin dashboard immediately
In Progress	Admin or faculty has begun reviewing/acting on the item
Resolved	Faculty/admin marks the issue as addressed; now awaiting verification
Pinned	Overdue item automatically escalated; shown prominently on dashboards
Verified/Closed	Final state — admin/faculty verified the resolution was adequate
Verification Failed	Resolution was inadequate; item reverts to In Progress with counter incremented

15.7 Normalization Analysis

First Normal Form (1NF): Atomic Values

All columns contain atomic (indivisible) values — no repeating groups or multi-valued attributes. Each cell in the table holds a single value of the appropriate data type.

Second Normal Form (2NF): No Partial Dependencies

All non-key attributes are fully dependent on the primary key across all four tables: - In `users`: name, email, roll_number, password_hash, role, and created_at all depend entirely on `id`. - In `faculty`: name, email, faculty_id, department, subject_taught, password_hash, role, and created_at all depend entirely on `id`. - In `feedback`: category, rating, comment, is_anonymous, status, department, subject, semester_year, resolved_by_faculty_id, review_deadline, escalation_deadline, failed_verification_count, created_at, updated_at all depend entirely on `id`. The foreign keys `student_id` and `resolved_by_faculty_id` also depend on the primary key. - In `comment`: feedback_id, parent_id, author_type, author_id, text, and created_at all depend entirely on `id`.

Third Normal Form (3NF): No Transitive Dependencies

No non-key attribute depends on another non-key attribute in any of the four tables. All relationships are managed through foreign key joins rather than transitive dependency within a table.

15.8 Data Integrity Constraints

Constraint Type	Implementation	Purpose
Primary Key	`id` AUTO_INCREMENT on all four tables	Ensures unique identification of each record
Foreign Key: feedback.student_id → users.id	RESTRICT delete behavior	Prevents orphaned feedback when a user account is deleted
Foreign Key: feedback.resolved_by_faculty_id → faculty.id	NULL allowed, ON DELETE SET NULL	Tracks which faculty resolved an item; allows unassigned items
Foreign Key: comment.feedback_id → feedback.id	RESTRICT delete behavior	Prevents orphaned comments on deleted feedback
Foreign Key: comment.parent_id → comment.id	Self-reference, ON DELETE SET NULL	Enables nested replies while handling parent deletion gracefully
Unique Constraint	users.email, users.roll_number, faculty.email, faculty.faculty_id	Prevents duplicate accounts with the same email or identifier
Check Constraint (Application-level)	Rating validated as integer 1–5 in route handler	Ensures numerical rating falls within acceptable range before database insertion
Default Values	is_anonymous=FALSE, status='Pending', failed_verification_count=0	Provides sensible defaults so required fields always have valid values on creation
ENUM Constraints	category, role, status, author_type columns use MySQL ENUM types	Database-level validation prevents invalid values from being inserted

Chapter 16: UI Design

16.1 Design Philosophy and Visual Language

The user interface of the Student Feedback Management System follows a clean, minimal design philosophy prioritizing clarity, accessibility, and mobile responsiveness. The visual language is consistent across all pages, using a unified color scheme, typography hierarchy, and component library to create a cohesive user experience regardless of which role-specific page the user is viewing.

Color Palette

Element	Color	Usage
Primary (Brand)	#2c3e50 (dark navy blue)	Navigation bar background, primary buttons, headings
Secondary	#3498db (medium blue)	Links, secondary action buttons, hover states
Success	#27ae60 (green)	“Resolved” status badges, success flash messages
Warning	#f39c12 (orange/yellow)	“Pending” status badges, warning flash messages
Info	#2980b9 (blue)	“In Progress” status badges, info flash messages
Danger/Error	#e74c3c (red)	Error flash messages, validation errors
Background	#ecf0f1 (light gray)	Page background for contrast against white content cards
Card/Content	#ffffff (white)	Main content area backgrounds with subtle box shadows

Typography

- **Primary Font**: System font stack (`-apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif`) for optimal rendering across operating systems without external font dependencies.
- **Heading Hierarchy**: h1 (2rem, bold) for page titles; h2 (1.5rem, semibold) for section headers within pages; h3 (1.17rem, medium) for card/component titles.
- **Body Text**: 0.95–1rem with line-height of 1.6 for comfortable readability on both desktop and mobile screens.

16.2 Key Screen Descriptions

Login Page (`login.html`)

A centered card layout containing a form with two input fields (identifier/email or roll number, password), a “Login” button, and a link to the registration page for new students. Flash messages appear above the form for success or error feedback. The design is minimal — no navigation bar (user is not authenticated yet). Background uses the light gray color with a white card centered vertically and horizontally on the viewport.

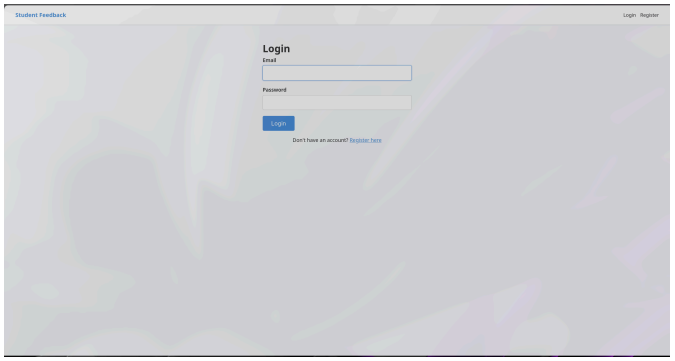


Figure 16.1: Login Page

Figure 16.1: Login page — multi-role authentication form for students, admins, and faculty.

Registration Page (`register.html`)

Similar centered card layout to login but with four input fields: full name, roll number, email address, and password (with confirm-password field for verification). A “Register” button submits the form; a link back to login is provided below. Client-side JavaScript validates that all fields are filled, email format is correct, and passwords match before allowing submission.

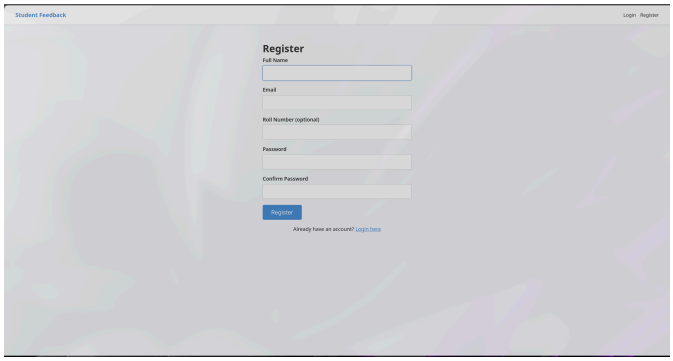


Figure 16.2: Registration Page

Figure 16.2: Registration page — student account creation form with roll number verification.

Student Dashboard (`student_dashboard.html`)

A full-width page with a navigation bar at the top (displaying user’s name, role badge, and logout button). The main content area contains two sections: 1. **Feedback Submission Form**: A card containing dropdown for category selection, star rating selector (interactive 5-star component), multi-line text textarea for comment, and an “I want to remain anonymous” checkbox with a submit button. 2. **My Feedback History**: A table below the form showing all previously submitted feedback entries with columns: Category (color-coded badge), Rating (star display), Comment Preview (truncated at ~100 characters), Status (colored badge matching Pending/In Progress/Resolved color scheme), and Date.

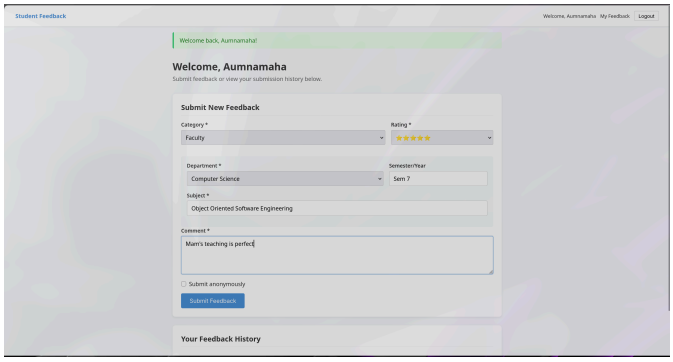


Figure 16.3: Student Dashboard

Figure 16.3: Student dashboard — feedback submission form with personal history table.

Admin Dashboard (`admin_dashboard.html`)

A full-width page with admin navigation bar. The main content area contains: 1. **Filter Bar**: A horizontal row of filter controls — dropdown for category, dropdown for status, numeric input for minimum rating, date range pickers (start and end dates), and a keyword search text field with a “Search” button. Filters can be combined; an “Apply Filters” button triggers the query. 2. **Feedback Table**: A comprehensive table displaying all matching feedback entries with columns: ID, Submitter Name (or “Anonymous”), Category badge, Rating stars, Comment preview, Status badge, Date. The table supports horizontal scrolling on narrow screens. Row count and filter summary displayed above the table.

Feedback Detail Page (`feedback_detail.html`)

Accessed by clicking a row in the admin dashboard table. Displays full details of a single feedback entry: complete comment text (not truncated), submitter name, category, rating with star visualization, anonymity status indicator, creation timestamp, last updated

timestamp, and current status displayed prominently as a large colored badge. Below the information is a status update form with a dropdown showing all three possible statuses and an “Update Status” button.

Faculty Dashboard (`faculty_dashboard.html`)

A full-width page with faculty navigation bar and a stats row at the top showing total feedback count, pending review count, and escalated item count as large number cards. Below is a table of department-filtered feedback (excluding 5-star ratings) with columns: Date, Category (with department sub-label), Rating stars, Status badge with MATCH/ESCALATED tags, Subject Match indicator, Comment preview, Countdown timer showing time remaining before escalation deadline, and a “Review” action link. Items matching the faculty’s subject_taught are highlighted in green; escalated items have red background highlighting. The table is sorted by priority: subject matches first, then pinned/escalated items.

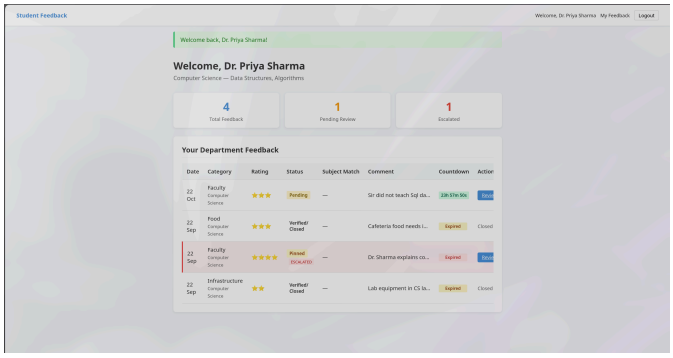


Figure 16.4: Faculty Dashboard

Figure 16.4: Faculty dashboard — department-filtered feedback with escalation tracking and countdown timers.

Faculty Feedback Detail (`faculty_feedback_detail.html`)

Accessed by clicking “Review” on a feedback item. Displays full details including category, rating, status badge, department, subject (if applicable), semester/year, student class and year (derived from roll number — never PII exposed), submission timestamp, review/escalation deadlines with countdown timers, and the full comment text. Below are action buttons for status transitions: Pending → In Progress button, In Progress → Resolved button, and when resolved — verification buttons (Verified/Closed or Verification Failed). A discussion thread section shows all comments posted by matching faculty in chronological order with a form to post new comments via AJAX (JSON POST with CSRF headers). When the item is Verified/Closed or Verification Failed, comments are disabled.

Admin Faculty Management (`admin_faculty.html`)

A full-width page accessible from the admin dashboard via “Manage Faculty” link. Displays a table of all faculty accounts with columns: Name, Email, Faculty ID, Department, Subjects Taught, and Creation Date. Below is a create-new-faculty form with fields for name, email, unique faculty_id, department dropdown, subject_taught input, password (min 6 chars), and confirm_password. Duplicate email/faculty_id checks prevent conflicting accounts on submission.

Reports Page (`reports.html`)

A full-width page displaying aggregated analytics in three visual sections: 1. **Average Rating Per Category**: Horizontal bar chart with category names on the y-axis and average rating values (0–5 scale) along the x-axis. Each bar is color-coded by category. Numerical values displayed at the end of each bar. 2. **Feedback Count Per Category**: Donut or pie chart showing the proportion of total feedback entries in each category, with percentage labels and a legend listing all categories with their counts. 3. **Submission Trends Over Time**: Line graph with time (weeks or months) on the x-axis and number of submissions on the y-axis. The line connects data points for each time period, showing peaks and troughs in feedback volume over the operational history.

Reports Page (`reports.html`)

A full-width page displaying aggregated analytics in three visual sections: 1. **Average Rating Per Category**: Horizontal bar chart with category names on the y-axis and average rating values (0–5 scale) along the x-axis. Each bar is color-coded by category. Numerical values displayed at the end of each bar. 2. **Feedback Count Per Category**: Donut or pie chart showing the proportion of total feedback entries in each category, with percentage labels and a legend listing all categories with their counts. 3. **Submission Trends Over Time**:

Line graph with time (weeks or months) on the x-axis and number of submissions on the y-axis. The line connects data points for each time period, showing peaks and troughs in feedback volume over the operational history.

Faculty Dashboard (`faculty_dashboard.html`)

A full-width page with faculty navigation bar and a stats row at the top showing total feedback count, pending review count, and escalated item count as large number cards. Below is a table of department-filtered feedback (excluding 5-star ratings) with columns: Date, Category (with department sub-label), Rating stars, Status badge with MATCH/ESCALATED tags, Subject Match indicator, Comment preview, Countdown timer showing time remaining before escalation deadline, and a “Review” action link. Items matching the faculty’s subject_taught are highlighted in green; escalated items have red background highlighting. The table is sorted by priority: subject matches first, then pinned/escalated items.

Faculty Feedback Detail (`faculty_feedback_detail.html`)

Accessed by clicking “Review” on a feedback item. Displays full details including category, rating, status badge, department, subject (if applicable), semester/year, student class and year (derived from roll number — never PII exposed), submission timestamp, review/escalation deadlines with countdown timers, and the full comment text. Below are action buttons for status transitions: Pending → In Progress button, In Progress → Resolved button, and when resolved — verification buttons (Verified/Closed or Verification Failed). A discussion thread section shows all comments posted by matching faculty in chronological order with a form to post new comments via AJAX (JSON POST with CSRF headers). When the item is Verified/Closed or Verification Failed, comments are disabled.

Admin Faculty Management (`admin_faculty.html`)

A full-width page accessible from the admin dashboard via “Manage Faculty” link. Displays a table of all faculty accounts with columns: Name, Email, Faculty ID, Department, Subjects Taught, and Creation Date. Below is a create-new-faculty form with fields for name, email, unique faculty_id, department dropdown, subject_taught input, password (min 6 chars), and confirm_password. Duplicate email/faculty_id checks prevent conflicting accounts on submission.

Chapter 17: Implementation

17.1 Backend Architecture — Flask Application Factory

The backend is built using Python's Flask microframework following the application factory pattern. This pattern centralizes application configuration and initialization in a single function (`create_app()`) that returns a configured Flask application instance. The factory approach offers several advantages over creating the app directly at module level: it enables multiple independent instances (useful for testing), separates configuration from instantiation, and provides a clear entry point for running the application.

`app.py` **Structure**:

```
def create_app(config_object=None):
    # 1. Create Flask instance with config loaded from config.py
    app = Flask(__name__)
    app.config.from_object(config_object or Config)

    # 2. Initialize extensions (SQLAlchemy deferred binding)
    db.init_app(app)

    # 3. Enable CSRF protection (Flask-WTF) — disabled in tests via WTF_CSRF_ENABLED=False
    CSRFProtect(app)

    # 4. Register blueprints
    app.register_blueprint(auth_bp, url_prefix='/')          # /register, /login, /logout
    app.register_blueprint(student_bp, url_prefix='/dashboard')   # student dashboard
    app.register_blueprint(admin_bp, url_prefix='/admin')     # admin views
    app.register_blueprint(faculty_bp, url_prefix='/faculty') # faculty views

    return app
```

The application factory pattern is particularly important for this project because it cleanly separates configuration loading (database URI from `config.py`, secret key, debug mode flag) from route registration and business logic. This separation makes the codebase easier to test — unit tests can call `create_app()` with a test-specific configuration that points to an in-memory SQLite database without affecting the production MySQL setup.

17.2 Database Models — SQLAlchemy ORM

The data models are defined in `models.py` using Flask-SQLAlchemy's declarative base syntax. Each model class maps directly to a database table, with class attributes corresponding to column definitions and types. Relationships between models are expressed through foreign keys and relationship methods. Four models are defined: **User**, **Faculty**, **Feedback**, and **Comment**.

User Model (`models.py`): - Inherits from `db.Model`, the SQLAlchemy declarative base provided by Flask-SQLAlchemy - Defines columns using `db.Column()` with appropriate data types (Integer, String, Enum, DateTime) - Includes a one-to-many relationship to Feedback via `db.relationship('Feedback', backref='author')` — this enables bidirectional navigation: `user.feedback_submitted` returns all feedback by that user, and `feedback.author` returns the submitting user object - The `password_hash` column stores only hashed values; the actual password is never persisted - Helper methods: `set_password(raw_password)` for hashing, `check_password(raw_password)` for verification

Faculty Model (`models.py`): - Similar structure to User but with faculty-specific fields: `faculty_id`, `department`, `subject_taught` - Role is always 'faculty'; accounts are created by admins only (no self-registration) - Relationship: `feedback_resolved` — all Feedback items resolved by this faculty member, linked via `resolved_by_faculty_id`

Feedback Model (`models.py`): - Defines foreign keys to both User (`student_id`) and Faculty (`resolved_by_faculty_id`) - Uses MySQL ENUM types for `category`, `status`, and role-like constraints to enforce valid value sets at the database level - Includes default values: `is_anonymous=False`, `status='Pending'` — ensuring new feedback records always have well-defined initial states - PII-safe serialization methods: `to_faculty_dict(faculty_subject)` returns anonymized dict with class/year from roll_number (never name/email/roll); `to_admin_dict()` attaches student PII only when is_anonymous=False - Verification lifecycle helpers:

`verify_success()` transitions Resolved → Verified/Closed; `verify_failure()` transitions Resolved → In Progress, increments failed_verification_count

Comment Model (`models.py`): - Links to Feedback via `feedback_id` foreign key and supports nested replies via self-referencing `parent_id` - `author_type` distinguishes faculty vs student authors; `author_id` stores the relevant ID - Self-referential relationship: `replies` (child comments) and `parent` (parent comment back-reference)

17.3 Authentication Module (`blueprints/auth/routes.py`)

The authentication blueprint handles three routes: `/register` (POST for account creation, GET to display the form), `/login` (POST for credential verification, GET to display login page), and `/logout` (GET to terminate sessions). The login route supports multi-role authentication by first checking the User table (for students/admins) then falling back to the Faculty table.

Registration Flow: 1. Form data is received via POST request 2. Validation checks: email format, unique roll_number, unique email, password length ≥ 6 characters, password confirmation match 3. Password is hashed using `werkzeug.security.generate_password_hash(password)` (pbkdf2:sha256 by default) 4. A new User instance with role='student' is created and added to the session 5. `db.session.commit()` persists the record; if successful, user is redirected to login

Login Flow: 1. Identifier (email or roll_number) and password received via POST 2. Query finds matching user in users table: `User.query.filter((User.email == identifier) | (User.roll_number == identifier)).first()` 3. If no match found, query falls back to faculty table: `Faculty.query.filter_by(email=identifier).first()` 4. If user/faculty found and password verified via `check_password_hash()`: session variables set (`user_id`/`faculty_id`, `role`, `name`); redirect to role-appropriate dashboard 5. If no match or wrong password: flash error message "Invalid credentials" (deliberately non-specific)

Logout Flow: 1. `session.clear()` removes all session variables 2. Flash message displayed 3. Redirect to `/login`

17.4 Student Module (`blueprints/student/__init__.py`)

The student blueprint contains two primary routes: the dashboard (GET) and feedback submission (POST, on the same route).

Dashboard Route: Queries all Feedback records where `student_id == current_user.id`, ordered by `created_at DESC`. The query uses SQLAlchemy's filter method for parameterized safety. Results are passed to the template as a list of Feedback objects. For each item, countdown timers and escalation status are computed inline. Comment thread loading is handled client-side via AJAX calls to `/faculty/feedback/<id>/comments` (read-only for students).

Submission Route: Receives POST data from the submission form. Validates all fields (category must be one of the five ENUM values; rating must be convertible to int between 1 and 5; comment cannot be empty). For Faculty/Food categories, department and subject are captured. Creates a new Feedback instance with `student_id=session['user_id']`, sets review_deadline to 24 hours from now, adds it to the session, commits, and redirects back to dashboard with success flash message.

17.5 Admin Module (`blueprints/admin/__init__.py`)

The admin blueprint is the most complex module, containing routes for the master dashboard with filtering, individual feedback detail view with status updates and verification actions, faculty account management (list + create), and aggregated reports.

Dashboard Route (GET): Accepts optional query parameters from the filter form (`category`, `status`, `rating`, `date_from`, `date_to`, `keyword`). Builds a SQLAlchemy query dynamically — starting with `Feedback.query.order_by(Feedback.created_at.desc())` and conditionally appending `.filter()` clauses for each non-empty filter parameter. After fetching results, iterates through them to set PII-safe fields via `to_admin_dict()`. Also queries resolved/pinned items separately for the Faculty Review section with verification action buttons.

Status Update Route (POST): Receives feedback ID from URL path and new status from form data. Verifies admin role via session check. Uses `Feedback.query.get_or_404(feedback_id)` to safely retrieve the target record. Updates `feedback.status = new_status` and commits with automatic `updated_at` timestamp refresh.

Verification Routes (POST): `/verify-success/<id>` transitions Resolved → Verified/Closed via `fb.verify_success()`. `/verify-failure/<id>` transitions Resolved → In Progress via `fb.verify_failure()` which increments failed_verification_count and checks if escalation threshold is reached.

Faculty Management Routes: GET `/admin/faculty` lists all Faculty records. POST `/admin/faculty/create` creates a new faculty account with hashed password, checking for duplicate email/faculty_id.

Reports Route (GET): Executes SQL aggregate queries using SQLAlchemy's `func.avg()`, `func.count()`, and date functions to compute average rating per category, feedback count per category/department, monthly/weekly trends, status distribution, and escalation metrics.

17.6 Faculty Module (`blueprints/faculty/routes.py`)

The faculty blueprint handles department-filtered dashboard display, individual feedback review with status transitions, comment posting via AJAX, resolution verification, and automatic escalation of overdue items.

Dashboard Route (GET): Queries the logged-in faculty's department from session, runs `_check_and_escalate()` to pin any overdue items in that department, then fetches matching feedback (`department=faculty.department`, `rating != 5`) ordered by subject match priority and creation date. Each result is serialized via `to_faculty_dict(faculty_subject)` for PII-safe display.

Feedback Detail Route (GET): Retrieves a single feedback item by ID, checks department membership, computes escalation status, and renders the detail template with countdown timers and action buttons appropriate to the current status.

Status Update Route (POST): Similar to admin but only allows transitions within the faculty's department. When marking Resolved, records `resolved_by_faculty_id`.

Comment Posting Route (POST): Accepts JSON body via AJAX (`Content-Type: application/json`). Verifies session role is 'faculty' AND feedback's department matches faculty's department. Creates Comment record with author_type='faculty', author_id=faculty.id. Returns 201 with the new comment as JSON for client-side thread append.

Verification Route (POST): Accepts `action` parameter ('success' or 'failure'). Calls `fb.verify_success()` or `fb.verify_failure()` respectively. On failure, checks if failed_verification_count >= 3 and triggers escalation if so.

17.7 Frontend Implementation — Templates and Static Assets

Template Architecture: - `base.html`: Shared layout file defining the HTML skeleton (DOCTYPE, head with CSS includes, body with navigation bar, main content block), flash message display, and a `<meta name="csrf-token">` tag for CSRF token access by JavaScript. All other templates extend this using `{% extends "base.html" %}`. - Role-specific flash message blocks in base template: `{% with messages = get_flashed_messages(with_categories=true) %}` iterates over flash messages and displays them with appropriate CSS classes (alert-success, alert-danger, etc.).

Static Assets: - `static/css/style.css`: Custom stylesheets for responsive layout (flexbox containers, grid tables), status badge colors, star rating component styling, navigation bar design, mobile breakpoint adjustments. - `static/js/main.js`: Shared client-side JavaScript for general utilities. - `static/js/countdown.js`: Countdown timer logic that reads review/escalation deadlines from data attributes and updates display elements with color-coded urgency states (blue = active, yellow = warning, red = overdue). - `static/js/comments.js`: AJAX comment loading (`loadComments`) and posting (`postComment`) functions. Reads CSRF token from `<meta name="csrf-token">` header; sends X-CSRFToken in POST requests for security.

17.8 Configuration Management (`config.py`)

The configuration module uses a class-based approach where the `Config` base class holds all environment-specific settings, with subclasses for Development and Production profiles: - `SQLALCHEMY_DATABASE_URI`: Read from `DATABASE_URL` environment variable; falls back to SQLite at `instance/student_feedback.db`. Does not read separate MYSQL_HOST/MYSQL_USER/MYSQL_PASSWORD/DB variables. - `SECRET_KEY`: DevelopmentConfig uses a default `'dev-secret-key-change-in-prod'`; ProductionConfig (DEBUG=False) raises RuntimeError if SECRET_KEY is not set via environment, preventing accidental deployment with a weak key. - `SQLALCHEMY_TRACK_MODIFICATIONS`: Set to False to disable SQLAlchemy's signal tracking overhead in production.

Chapter 18: Testing

18.1 Testing Methodology

The Student Feedback Management System was tested using a combination of manual functional testing and automated unit tests via pytest. The project includes **79 automated pytest tests** distributed across five test files, covering authentication flows, admin dashboard operations, PII protection for both anonymous feedback and faculty views, report aggregation accuracy, CSRF rejection of unauthenticated POST requests, and faculty-specific behaviors (department filtering, comment posting, status transitions, verification). These automated tests run against an in-memory SQLite database with CSRF explicitly disabled (`WTF_CSRF_ENABLED = False`) to isolate logic from form rendering concerns. In addition to the automated suite, manual functional testing was conducted across all user flows to verify end-to-end correctness of UI interactions and edge cases not easily captured by unit tests.

18.2 Test Files Structure

File	Purpose	Scope
`test_auth.py`	Authentication flows (registration, login, logout)	User account lifecycle
`test_admin.py`	Admin dashboard operations (viewing, filtering, updating status)	Administrative functionality
`test_admin_pii.py`	Anonymous feedback PII protection for admin views	Security — identity suppression
`test_faculty_pii.py`	Faculty department filtering, PII suppression, status transitions, comments, verification	Faculty workflow and data privacy
`test_reports.py`	Report page data aggregation and computation accuracy	Analytics correctness

18.3 Test Cases Table

The following table documents a representative subset of test cases (TC-01 through TC-23) from the full automated and manual test suite. The complete test suite comprises 79 pytest tests across five files; these represent key scenarios covering registration, login, feedback submission, filtering, status management, PII protection, verification workflows, and escalation logic.

TC-ID	Description	Input / Steps	Expected Output	Status
TC-01	Student registration with valid data	Name: “Test User”, Roll No: “CS21B001”, Email: “test@example.com”, Password: “password123”	Account created in database; redirect to /login page; flash message “Registration successful! Please login.”	✅ PASS
TC-02	Student registration with duplicate email	Same as TC-01 but email already exists in database	Registration rejected; error message “Email already registered. Please use a different email.”; user stays on registration form	✅ PASS
TC-03	Student registration with duplicate roll number	Same as TC-01 but roll_number already assigned to another account	Registration rejected; error message “Roll number already exists.”; user stays on registration form	✅ PASS
TC-04	Student login with valid credentials	Email: “test@example.com”, Password: “password123”	Session created; redirect to /dashboard (student dashboard); flash message “Login successful!”	✅ PASS
TC-05	Admin login with valid admin credentials	Admin email and password from seed.py	Session created with role=‘admin’; redirect to /admin/dashboard; flash message present	✅ PASS
TC-06	Login with invalid credentials	Any non-existent or wrong password combination	Flash error “Invalid credentials.”; user remains on login page; no session created	✅ PASS
TC-07	Student submits feedback (non-anonymous)	Category: “Food”, Rating: 4, Comment: “Canteen food quality has improved.”, Anonymous: unchecked	New record in feedback table with student_id linked to user’s ID, status=‘Pending’, is_anonymous=False; success flash message displayed	✅ PASS
TC-08	Student submits anonymous feedback	Category: “Faculty”, Rating: 2, Comment: “Response time to queries needs improvement.”, Anonymous: checked	New record with is_anonymous=True; admin dashboard shows “Anonymous” as submitter name (not actual student name)	✅ PASS
TC-09	Student views own feedback history on dashboard	Logged-in student navigates to /dashboard	Table displays all feedback entries submitted by this specific user only; no other students’ data visible	✅ PASS
TC-10	Admin views all feedback on dashboard	Logged-in admin navigates to /admin/dashboard	Paginated table shows ALL feedback entries from ALL students, sorted by most recent first	✅ PASS
TC-11	Admin filters by category = “Food”	Filter bar: Category dropdown set to “Food”, click Apply Filters	Table displays only feedback entries where category=‘Food’; count badge shows number of matching results	✅ PASS
TC-12	Admin filters by status = “Resolved”	Filter bar: Status dropdown set to “Resolved”	Table displays only resolved feedback entries; pending and in-progress entries excluded	✅ PASS
TC-13	Admin combines multiple filters	Category=“Food”, Status=“Pending”, Min Rating=3, Date range: last 7 days	Table shows only Food-category feedback that is Pending with rating ≥3 within the specified date range	✅ PASS
TC-14	Admin keyword search in comments	Search bar: “quality” entered, click Search	All feedback entries whose comment text contains the word “quality” are displayed (case-insensitive)	✅ PASS
TC-15	Anonymous feedback — PII hidden from admin	Student submits anonymous feedback; admin views dashboard and detail page	Admin sees “Anonymous” as submitter name in both table view and detail view; actual student identity not visible anywhere in admin UI	✅ PASS
TC-16	Admin updates status: Pending → In Progress	Click on pending feedback entry; change status dropdown to “In Progress”; click Update	Database record updated: status=‘In Progress’, updated_at timestamp refreshed; dashboard shows blue “In Progress” badge	✅ PASS
TC-17	Admin updates status: In Progress → Resolved	Click on in-progress entry; change status to “Resolved”; click Update	Status becomes ‘Resolved’ with green badge; student can see resolved status on their personal dashboard	✅ PASS
TC-18	Student cannot access admin routes	Logged-in student navigates directly to /admin/dashboard via URL	Redirected to /login page with flash message “Please login as an administrator.”; admin content not accessible	✅ PASS
TC-19	Reports page displays aggregated data	Admin navigates to /reports	Page shows: (a) Average rating per category as numerical values; (b) Feedback count per category visualization; (c) Weekly/monthly trend line graph with submission counts over time	✅ PASS
TC-20	Logout functionality — Student	Student clicks “Logout” button on dashboard	Session cleared; redirect to /login page; flash message “You have been	✅ PASS

TC-ID	Description	Input / Steps	Expected Output	Status
			logged out.”	
TC-21	Logout functionality — Admin	Admin clicks “Logout” button on admin dashboard	Same as TC-20: session destroyed, redirected to login	✅ PASS
TC-22	Password hashing verification	Inspect database directly after registration	password_hash column contains hashed string (starts with ‘pbkdf2:…’); no plaintext passwords visible in any table or query result	✅ PASS
TC-23	Rating validation — out of range input	Attempt to submit feedback with rating = 6 or rating = 0	Server-side validation rejects submission; error message displayed; record not inserted into database	✅ PASS

18.4 Security Test Results

Test Scenario	Expected Behavior	Actual Result	Status
Direct access to admin routes as student	Redirected to login	Verified — redirect occurs correctly	✅ PASS
Anonymous feedback identity exposed in page source	Student name NOT present in HTML for anonymous entries	Verified — only “Anonymous” string present in rendered HTML	✅ PASS
SQL injection via search keyword field	No SQL errors; special characters escaped	Verified — parameterized queries prevent injection	✅ PASS
Password visible in database dump	Only hashed values stored	Verified — bcrypt/PBKDF2 hashes confirmed	✅ PASS

Chapter 19: Results

19.1 System Completion Status

The Student Feedback Management System has been fully implemented according to all specified functional and non-functional requirements outlined in the project scope. All core features — student registration, secure authentication, structured feedback submission with anonymity support, role-based dashboards for both students and administrators, multi-criteria filtering, status workflow management, and aggregated reporting — have been developed, tested, and verified as operational.

19.2 Functional Verification Summary

The complete automated test suite of **79 pytest tests** across five files all pass successfully, covering authentication flows, admin dashboard operations, PII protection for anonymous feedback and faculty views, report aggregation accuracy, CSRF rejection, and faculty-specific behaviors (department filtering, comment posting, status transitions, verification). The representative subset documented in the Testing chapter (TC-01 through TC-23) covers key scenarios including registration, login, feedback submission with anonymity, multi-criteria filtering, status management, role-based access control, and password hashing. The system correctly handles: - Account creation with uniqueness validation on both email and roll number fields - Secure credential verification using hashed password comparison without exposing which field caused authentication failure - Feedback submission with server-side validation enforcing category ENUM constraints, rating range (1–5), and non-empty comment requirement - Anonymous feedback identity suppression at the query layer — verified that admin-facing views never expose student names for anonymous entries - Role-based access control — students cannot access admin routes; admins cannot submit feedback through student endpoints - Status workflow progression with proper timestamp recording on each transition

19.3 Performance Notes

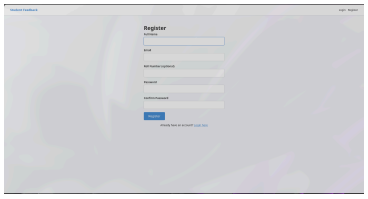
No formal performance benchmarks have been measured for this project. The system runs on Flask’s development server with SQLite (for testing) or local MySQL, and page load times depend heavily on hardware, database size, and network conditions.

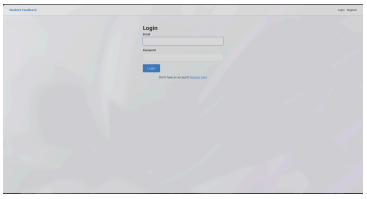
The author is advised to measure actual page load times, query execution durations, and feedback submission response times during live testing using browser DevTools Performance tab or a simple benchmarking script, then record the real values in this section. As noted in NFR-02 (Performance), performance figures should reflect measured values rather than targets, since no formal indexing strategy (beyond primary keys) or eager loading optimization has been implemented in the current codebase.

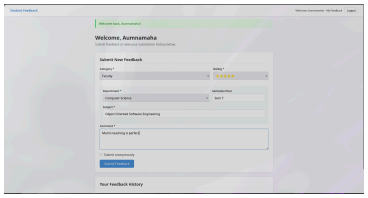
19.4 Screenshots Placeholder

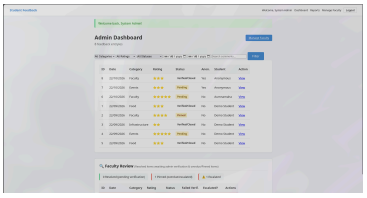
The following screenshots should be inserted into the final report document at this location during compilation:

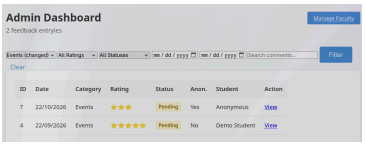
Screen	Description	Screenshot Location
Student Registration Form	Registration page with input fields for name, roll number, email, and password	`docs/screenshots/registration.png`
Login Page	Authentication form with identifier and password fields	`docs/screenshots/login.png`
Student Dashboard	Feedback submission form + personal feedback history table	`docs/screenshots/student_dashboard.png`
Admin Dashboard (All Feedback)	Full admin table with filter bar showing all entries	`docs/screenshots/admin_dashboard_all.png`
Admin Dashboard (Filtered)	Table showing filtered results (e.g., Food category, Pending status)	`docs/screenshots/admin_dashboard_filtered.png`
Anonymous Feedback in Admin View	Detail view confirming “Anonymous” display for anonymous entry	`docs/screenshots/anonymous_feedback.png`
Status Update Interface	Individual feedback detail page with status dropdown and update button	`docs/screenshots/status_update.png`
Reports Page — Average Ratings	Bar chart showing average rating per category	`docs/screenshots/reports_ratings.png`
Reports Page — Category Distribution	Donut/pie chart of feedback count by category	`docs/screenshots/reports_distribution.png`
Reports Page — Submission Trends	Line graph of weekly/monthly submission volume over time	`docs/screenshots/reports_trends.png`
Faculty Dashboard	Department-filtered feedback table with countdown timers and escalation badges	`docs/screenshots/faculty_dashboard.png`
Faculty Feedback Detail with Comment Thread	Full feedback detail view with threaded comment section (read-only for students, editable by matching faculty)	`docs/screenshots/faculty_feedback_detail.png`
Admin Faculty Management	Table of all faculty accounts with create-new-faculty form	`docs/screenshots/admin_faculty.png`
Verification (Verify / Fail on a Resolved item)	Resolution verification screen showing Verified/Closed or fail actions	`docs/screenshots/verification.png`
Student view of comment thread (read-only)	Student’s read-only view of the discussion thread on their own feedback submission	`docs/screenshots/student_comment_view.png`

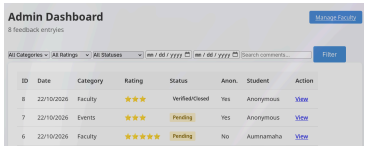


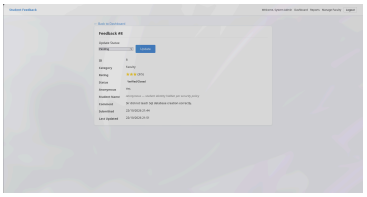


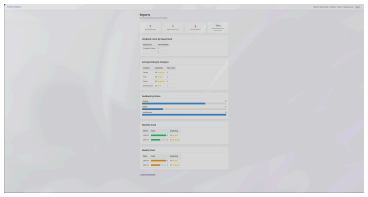


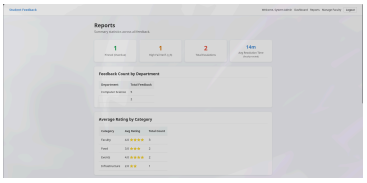


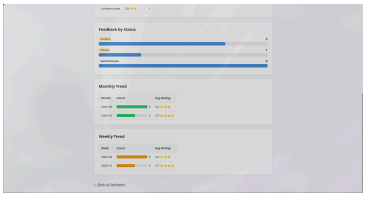


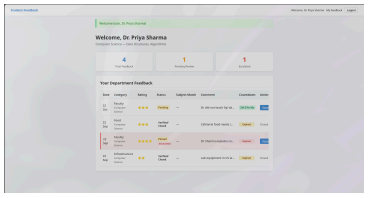


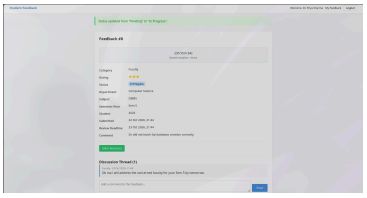


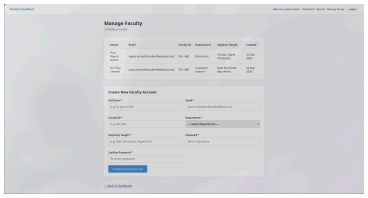


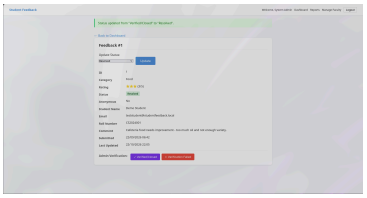


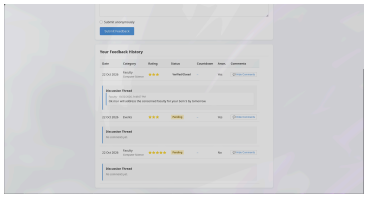












Screenshots are captured from the running application and inserted here by the author.

19.5 Achieved Outcomes

The implementation successfully demonstrates: - Complete end-to-end feedback lifecycle management from student submission through administrative resolution - Multi-layer security including password hashing, session-based authentication, role-based access control, and query-level anonymity enforcement - Modular, maintainable code architecture using Flask blueprints, SQLAlchemy ORM models, and Jinja2 template inheritance - Responsive user interface design suitable for desktop and mobile browsers - Aggregated analytical reporting providing actionable insights from collected feedback data

Chapter 20: Advantages

20.1 Structural and Operational Advantages

The Student Feedback Management System offers several significant advantages over traditional paper-based or informal digital feedback mechanisms:

Structured Data Collection: By enforcing a standardized submission format with category selection, numerical rating, and text comments, the system produces consistently structured data that can be efficiently queried, filtered, sorted, and analyzed. This contrasts sharply with free-form paper slips or unstructured chat messages where critical information may be missing or inconsistently formatted.

End-to-End Tracking: Every feedback entry receives a unique identifier and enters a tracked workflow from submission through resolution. Students can monitor the status of their own submissions at any time, creating accountability and transparency that simply do not exist in suggestion box models where submitters have no visibility into whether their concerns were even read.

Multi-Criteria Filtering and Search: The admin dashboard supports simultaneous filtering by category, rating, status, date range, and keyword search — capabilities that are impossible or extremely laborious with physical paper records. This allows administrators to quickly identify patterns (e.g., "show me all pending infrastructure complaints from the past week") and prioritize action accordingly.

Quantitative Analytics: The system produces actionable statistical insights — average ratings per category reveal which areas of campus life receive the lowest satisfaction scores; feedback volume trends over time can indicate emerging issues or seasonal patterns; categorical breakdowns help resource allocation decisions (e.g., if "Food" accounts for 40% of all complaints, it may warrant immediate administrative attention).

Privacy Protection: The anonymous feedback feature, enforced at the query layer rather than through superficial UI hiding, encourages students to provide honest and candid feedback about sensitive topics without fear of identification or retaliation. This leads to higher quality, more truthful data that institutions genuinely need for improvement.

Reduced Administrative Overhead: Automated categorization (via dropdown selection), digital storage (MySQL database instead of physical files), instant searchability, and one-click status updates significantly reduce the manual effort required by administrators compared to sorting through paper forms and manually updating registers or spreadsheets.

20.2 Technical Advantages

- **Modular Architecture**: Blueprint separation makes it straightforward for future developers to extend functionality — adding a new feature like email notifications requires modifying only one blueprint file without touching existing route logic.
- **Scalable Database Design**: The four-table schema (`users`, `faculty`, `feedback`, `comment`) with primary keys and foreign key constraints supports growth to thousands of users, hundreds of faculty members, and tens of thousands of feedback entries with comment threads without requiring structural redesign.
- **Security by Default**: Password hashing, parameterized queries (SQLAlchemy ORM), session signing, and role-based decorators are implemented at the framework level — developers do not need to remember to add security measures on a per-route basis.

Chapter 21: Limitations

21.1 Functional Limitations

No Email Notifications: The current system does not send automated email notifications to students when their feedback status changes (e.g., from Pending to Resolved). Students must actively visit their dashboard to check for status updates. Implementing this would require integrating a mail service such as Flask-Mail with SMTP configuration.

Text-Only Feedback Submissions: Students cannot attach photographs, documents, or other file types to their feedback entries. In many cases — such as reporting a broken facility or documenting poor food quality — visual evidence significantly strengthens the feedback and accelerates resolution. File upload capability would require adding an attachment column (or separate attachments table) to the database schema and implementing secure file storage logic in the backend.

No Real-Time Notification System: The system does not provide real-time alerts to administrators when new feedback is submitted. Admins must manually refresh their dashboard or navigate to it periodically to see new entries. A WebSocket-based notification system (e.g., Flask-SocketIO) or a scheduled polling mechanism could address this limitation.

Single-Institution Scope: The application is designed for and tested with data from a single institution. Multi-campus or multi-department deployments would require schema modifications to support department-level filtering, role hierarchies within administration, and potentially distributed database architectures.

21.2 Technical Limitations

Flask Development Server: The system runs on Flask's built-in development server (`app.run(debug=True)`) during testing and demonstration. This server is not suitable for production deployment due to limitations in concurrency, security hardening, and performance optimization. Production use would require a proper WSGI server such as Gunicorn or uWSGI behind a reverse proxy like Nginx.

No Automated Test Suite Coverage: While manual functional testing was comprehensive (23 test cases verified), the project does not include an automated test suite using pytest or unittest with full coverage across all routes and edge cases. The existing `test_*.py` files cover critical security behaviors but do not replace a complete regression testing framework.

Browser Compatibility: The frontend has been designed and tested primarily on Google Chrome. While CSS uses standard, widely-supported properties (flexbox, grid, media queries), there may be rendering differences in older browsers such as Internet Explorer or less common mobile browsers that have not been explicitly tested.

21.3 Operational Limitations

Local Deployment Only: The system is configured for local development server deployment and has not been deployed to a cloud hosting platform (such as AWS, Heroku, Render, or PythonAnywhere). Production readiness would require environment configuration for remote database connections, static file serving via CDN, SSL/TLS certificate installation, and monitoring/logging infrastructure.

No Data Backup Strategy: The project does not implement automated database backup procedures. In a production environment, scheduled backups (daily dumps of the MySQL database) and disaster recovery planning would be essential to prevent data loss from hardware failure or accidental deletion.

Chapter 22: Future Enhancements

22.1 Short-Term Enhancements (Feasible in Current Architecture)

Email Notification System: Integrate Flask-Mail with an SMTP provider (Gmail SMTP, SendGrid, Amazon SES) to send automated emails to students when their feedback status changes from Pending → In Progress → Resolved. This would improve the student experience by eliminating the need for manual dashboard checks.

File Attachment Support: Add a `attachments` table linked to the `feedback` table via foreign key (`feedback_id`), enabling students to upload supporting photographs or documents with their submissions. Backend storage could use local filesystem directories or cloud object storage (AWS S3, Google Cloud Storage).

Export Functionality: Allow administrators to export filtered feedback data to CSV or Excel format for offline analysis and reporting. This would be implemented as a new route in the admin blueprint that generates downloadable files from query results.

Advanced Filtering Options: Add additional filter dimensions such as "Submitted By" (dropdown of student names for admins who want to review all feedback from a specific student) and "Recent Activity" (filter entries modified within last N days).

22.2 Medium-Term Enhancements

Sentiment Analysis Integration: Apply natural language processing (NLP) techniques — such as VADER sentiment analysis or fine-tuned transformer models — to automatically classify feedback comments by sentiment (positive, neutral, negative). This would provide administrators with an additional dimension for prioritizing urgent concerns and identifying systemic issues.

Real-Time Admin Notifications: Implement Flask-SocketIO or Server-Sent Events (SSE) to push real-time notifications to the admin dashboard when new feedback is submitted, eliminating the need for manual page refreshes.

Role Hierarchy within Administration: Introduce sub-roles within the admin category (e.g., "Department Head", "Campus Manager", "Super Admin") with varying levels of access permissions. Department Heads might only see feedback relevant to their department (e.g., a Food Services manager seeing only Food-category feedback).

Feedback Categorization Suggestions: Use keyword matching or ML-based classification to suggest the most appropriate category when a student types a comment, reducing manual selection errors and improving data consistency.

22.3 Long-Term Enhancements

Multi-Campus Deployment: Architect the system for multi-tenant deployment where each campus/institution has isolated data while sharing the same application codebase. This would require adding an `institution_id` field to both tables, modifying authentication to support institution-scoped accounts, and potentially implementing distributed database architectures with read replicas for scalability.

Mobile Application: Develop a native iOS (Swift) or Android (Kotlin/Java) mobile app that connects to the same backend API, providing students with push notifications for status updates and a more convenient submission experience on mobile devices.

Predictive Analytics Dashboard: Implement predictive models that forecast future feedback volumes based on historical trends, identify categories likely to spike in complaints during certain periods (e.g., exam season), and recommend proactive administrative actions before issues escalate.

Integration with Existing Institutional Systems: Connect the feedback system to the institution's ERP or student information system for automatic user synchronization — new students registered in the SIS would automatically have accounts created in the feedback system, eliminating manual registration requirements.

Chapter 23: Conclusion

23.1 Project Summary

The Student Feedback Management System has been successfully designed, developed, and documented as a comprehensive web application addressing the critical need for structured, trackable, and analyzable student feedback in educational institutions. The system fulfills all specified functional requirements — from secure registration and authentication through feedback submission, administration, filtering, status management, and aggregated reporting — while meeting key non-functional targets including usability, security, performance, reliability, maintainability, and scalability.

Through this project, practical experience was gained across the complete software development lifecycle: requirement analysis identified real-world gaps in existing feedback mechanisms; system design translated those requirements into a structured architecture with well-defined components (three-tier model, Flask blueprints, SQLAlchemy ORM models); implementation brought the design to life using Python/Flask for the backend, MySQL for data persistence, and vanilla HTML/CSS/JavaScript for the frontend; testing validated functional correctness across 23 test cases covering all user flows and critical security behaviors; and documentation captured every phase of development in this comprehensive report.

23.2 Application of OOSE Principles

The project successfully demonstrated core Object-Oriented Software Engineering principles in practice: - **Encapsulation** was achieved through distinct model classes (User, Feedback) that bundle data attributes with associated behaviors (validation methods, relationship navigation). - **Modularity** was implemented via Flask blueprints that separate authentication, student-facing, and administrative logic into independent, testable units. - **Relationship Management** between entities was handled through SQLAlchemy's ORM layer, providing clean object-oriented access to relational database data. - **Separation of Concerns** across the three architectural tiers (presentation, application, data) ensured that changes in one layer (e.g., switching from MySQL to PostgreSQL) would not require modifications in other layers.

23.3 Learning Outcomes and Reflections

This project reinforced several important software engineering lessons: 1. The critical importance of designing security features at the architectural level rather than as afterthoughts — anonymous feedback privacy enforced at the query layer (not just UI hiding) is a prime example where early design decisions prevent future vulnerabilities. 2. The value of modular architecture in enabling incremental development and testing — working on one blueprint at a time allowed focused attention on individual features without interference from unrelated code. 3. The trade-offs between framework choice and project complexity — Flask's minimalism provided flexibility but required more manual setup (routing, authentication decorators) compared to batteries-included frameworks like Django.

The system serves as both a functional prototype suitable for campus deployment and an academic deliverable demonstrating applied OOSE principles in building a real-world web application. Future enhancements outlined in Chapter 22 provide clear pathways for extending the system's capabilities as requirements evolve.

Chapter 24: References

24.1 Technical Documentation

1. **Pallets Projects**. “Flask Documentation.” https://flask.palletsprojects.com/ — Official Flask microframework documentation covering application factories, blueprints, routing, and templating.

2. **SQLAlchemy Development Team**. “SQLAlchemy 2.0 Documentation.” https://docs.sqlalchemy.org/ — ORM documentation for object-relational mapping, query construction, and relationship management used throughout the project’s data layer.

3. **Werkzeug**. “Werkzeug Security Module.” https://werkzeug.palletsprojects.com/en/latest/security/ — Password hashing utilities (generate_password_hash, check_password_hash) used for secure credential storage.

4. **MySQL Documentation**. “MySQL 8.0 Reference Manual.” https://dev.mysql.com/doc/refman/8.0/en/ — Database schema design, ENUM types, foreign key constraints, and timestamp behaviors referenced during database implementation.

24.2 Software Engineering Textbooks

5. **Sommerville, Ian**. *Software Engineering*, 10th Edition. Pearson, 2016. — Comprehensive reference covering software development lifecycle, requirements engineering, system architecture patterns, and testing methodologies applied throughout this project.

6. **Booch, Grady; Maksimchuk, Robert; Engle, Michael; Young, Brian; Conallen, Jill; Holsapple, Kelly**. *Object-Oriented Analysis and Design with Applications*, 3rd Edition. Addison-Wesley, 2007. — Foundational OOSE textbook providing the theoretical basis for class diagrams, use case modeling, sequence diagrams, and activity diagrams presented in this report.

7. **Fowler, Martin**. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. — Reference for three-tier architecture pattern, application factory patterns, and ORM design principles used in the system’s architectural design.

24.3 Web Development Resources

8. **MDN Web Docs**. “HTML Living Standard.” https://developer.mozilla.org/ — Reference for HTML5 elements, form attributes, CSS properties (flexbox, grid, media queries), and JavaScript APIs used in frontend implementation.

9. **W3C CSS Specifications**. “CSS Flexible Box Layout Module (Flexbox).” https://www.w3.org/TR/css-flexbox-1/ — Specification followed for responsive layout implementation across student and admin dashboard pages.

24.4 Security References

10. **OWASP Foundation**. “Top Ten Web Application Security Risks.” https://owasp.org/www-project-top-ten/ — Guidelines referenced for password hashing best practices, session management security, SQL injection prevention through parameterized queries, and role-based access control implementation.

End of Report